

ChaNGa Computational Challenges for Cosmological Codes

Robert Wissing - University of Oslo

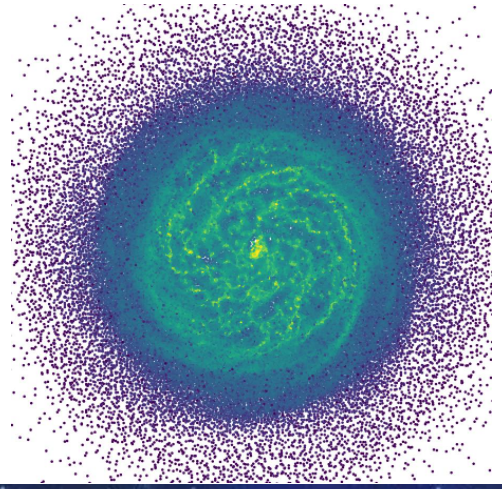
Co-funded by the European Union

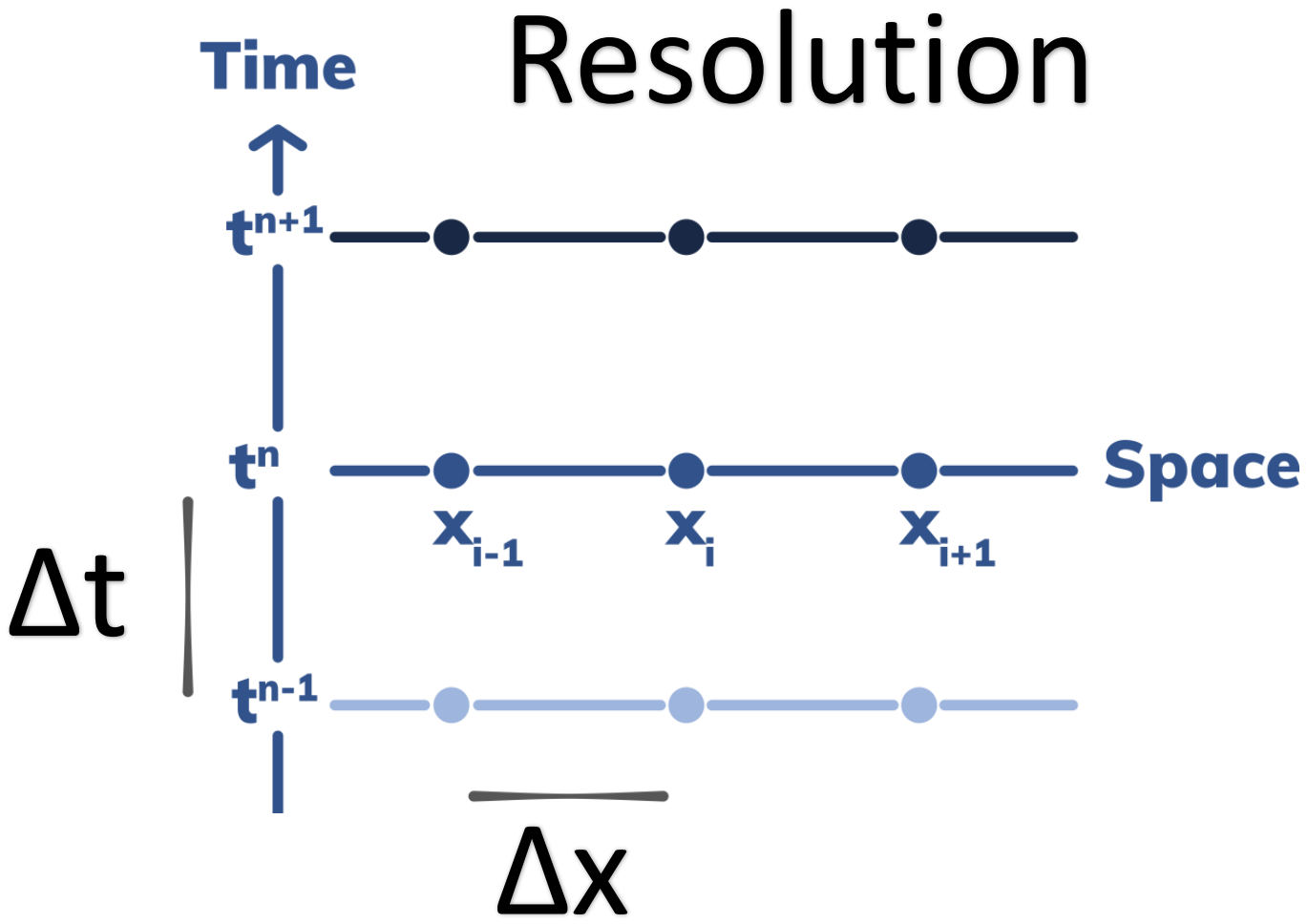
Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Belgium, Czech Republic, France, Germany, Greece, Italy, Norway, and Spain under grant agreement No 101093441.

EuroHPC Joint Undertaking

Numerical simulation – continuum -> discretized





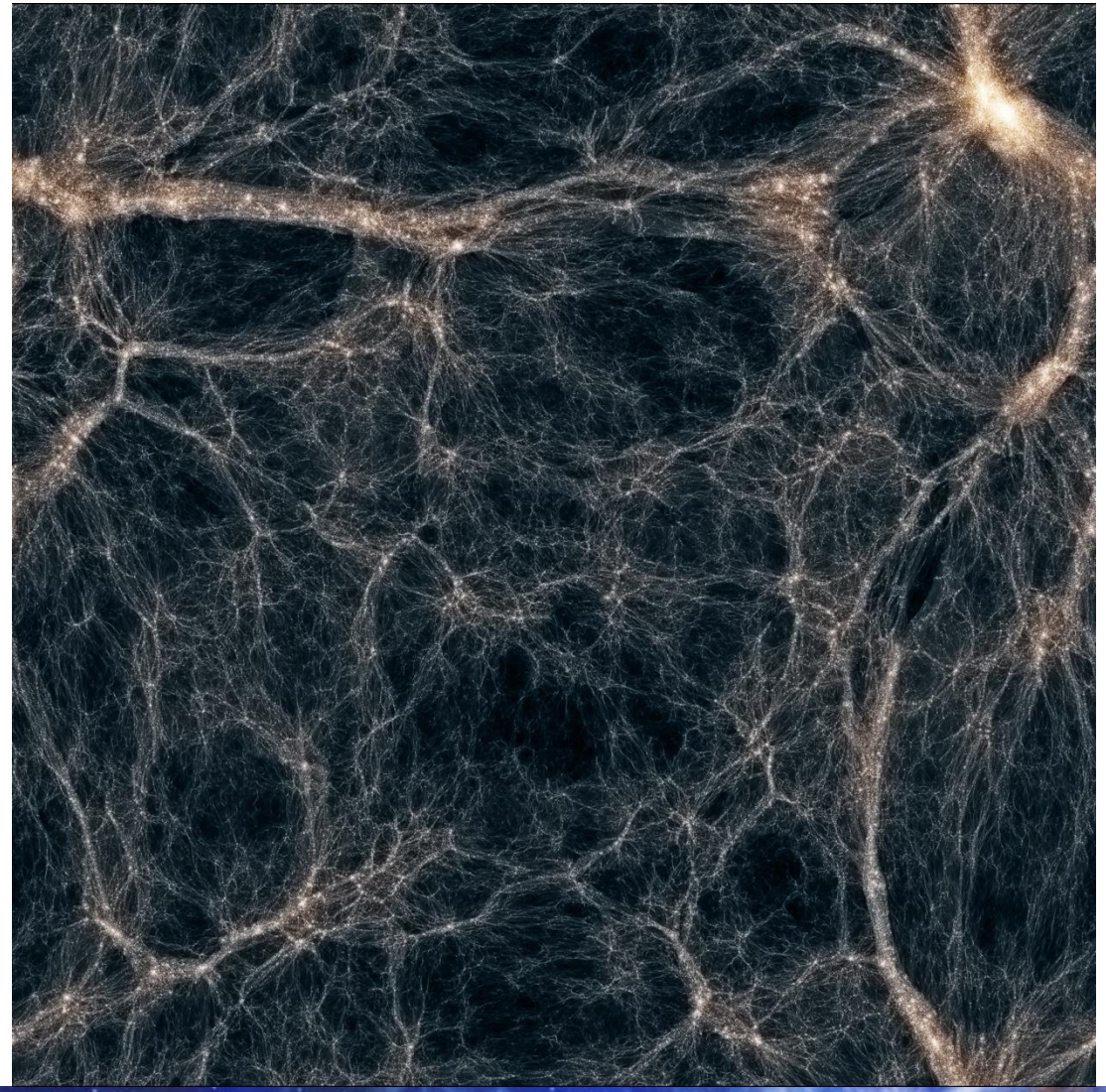


Cosmological Simulations

Cosmological box /
Patch of the universe

25Mpc

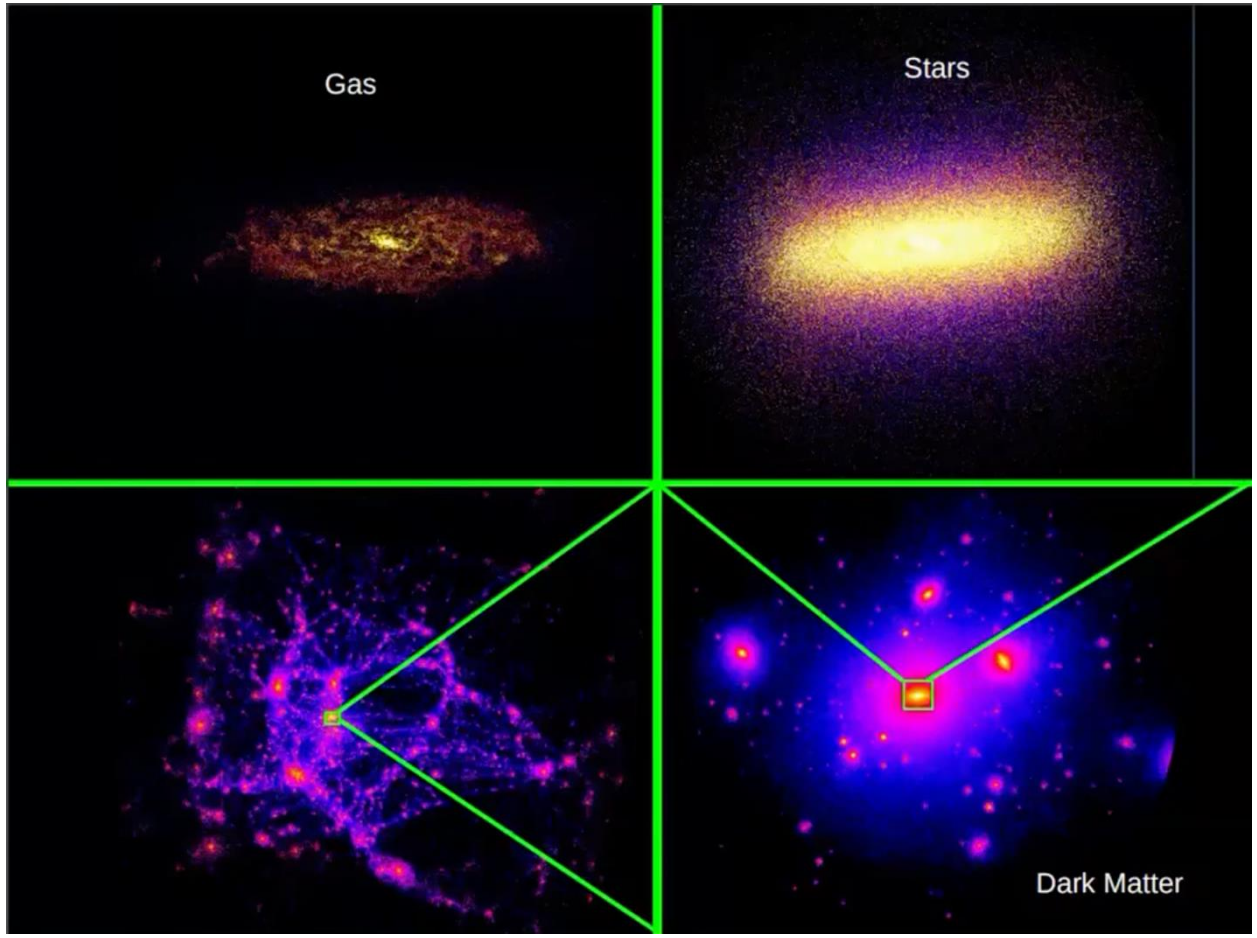
80 million lyrs



Computational challenges

Cosmological codes presents several challenges for scaling.

1. **Large spatial dynamic range**
   Distance to closest star ~1 pc
   Galaxy size in order of ~10kpc
   Distance between galaxies ~1Mpc

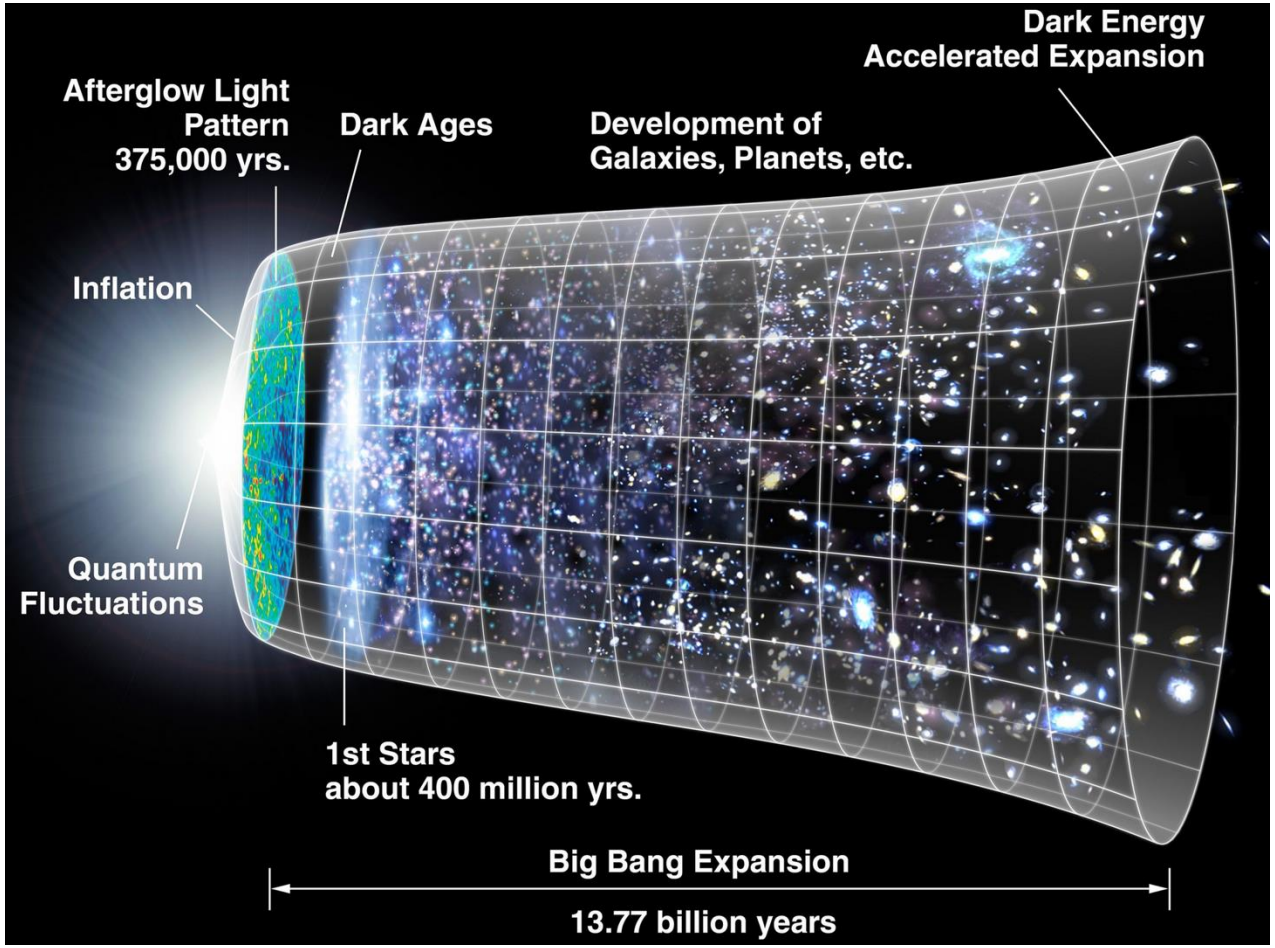


Computational challenges

Cosmological codes presents several challenges for scaling.

1. **Large spatial dynamic range**
   Distance to closest star ~1 pc
   Galaxy size in order of ~10kpc
   Distance between galaxies ~1Mpc

2. **Large temporal range (multi-time stepping)**
   age of universe ~10Gyr
   dynamic time in galaxies ~<1Myr



Computational challenges

Cosmological codes presents several challenges for scaling.

1. **Large spatial dynamic range**
   Distance to closest star ~1 pc
   Galaxy size in order of ~10kpc
   Distance between galaxies ~1Mpc
2. **Large temporal range (multi-time stepping)**
   age of universe ~10Gyr
   dynamic time in galaxies ~<1Myr
3. **A lot of physics!**
   Subgrid modules needed, when resolution is insufficient
4. **Different force ranges**

Most important astrophysical processes

Gas cooling	Interstellar medium	Star formation	Stellar feedback	Supermassive black holes	Active galactic nuclei	Magnetic fields	Radiation fields	Cosmic rays
Atomic/ molecular/ metals/ tabulated network	Effective equation of state/ multiphase	Initial stellar mass function/ probabilistic sampling/ enrichment	Kinetic/ thermal/ variety of sources from stars, supernovae	Numerical seeding/ growth by accretion prescription/ merging	Kinetic/ thermal/ radiative/ quasar model radio mode	Ideal MHD/ cleaning schemes/ constrained transport	Ray tracing/ Monte Carlo/ moment based	Production/ heating/ anisotropic diffusion/ streaming

Computational challenges

Cosmological codes presents several challenges for scaling.

1. **Large spatial dynamic range**
   Distance to closest star ~1 pc
   Galaxy size in order of ~10kpc
   Distance between galaxies ~1Mpc
2. **Large temporal range (multi-time stepping)**
   age of universe ~10Gyr
   dynamic time in galaxies ~<1Myr
3. **A lot of physics!**
   Subgrid modules needed, when resolution is insufficient
4. **Different force ranges**
5. **Different fluids/particle species**
   dark matter (gravity)
   gas (gravity, hydro, feedback, cooling)
   stars (gravity, feedback)

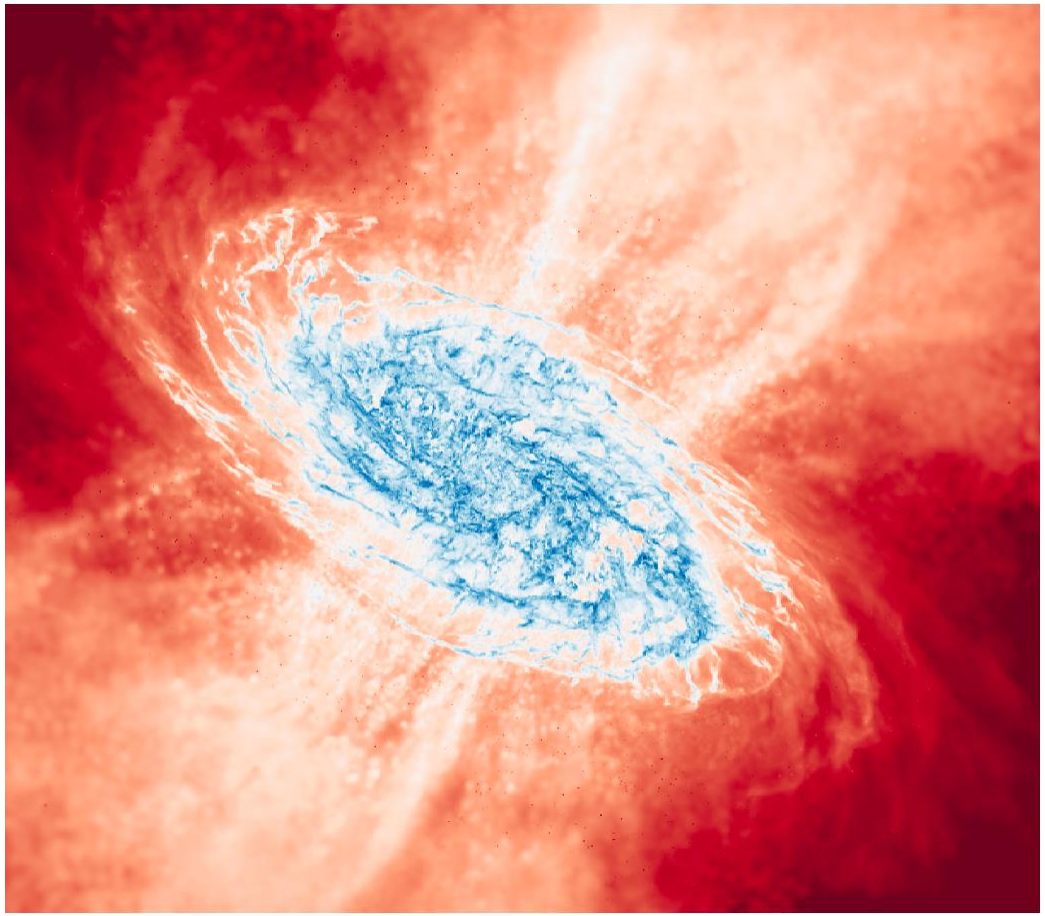
What is ChaNGa ?

Astrophysics code for cosmology, galaxy, star and planet formation

Five major physics algorithms within ChaNGa:

1. Tree-based gravity solver (N-body)
2. Smoothed-Particle Magnetohydrodynamics (SPMHD)
3. Star formation, stellar and AGN feedback
4. Cosmological boundary/Ewald summation
5. Radiative Cooling

ChaNGa: utilises Charm++ runtime system for drastically improved parallelisation



N-body – Discretization of gravity

Self-gravitating fluid:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + v\frac{\partial f}{\partial r} - \frac{\partial \Phi}{\partial r}\frac{\partial f}{\partial v} = 0$$

$$\nabla^2\Phi = 4\pi G \int f\,dv = 4\pi G\rho$$

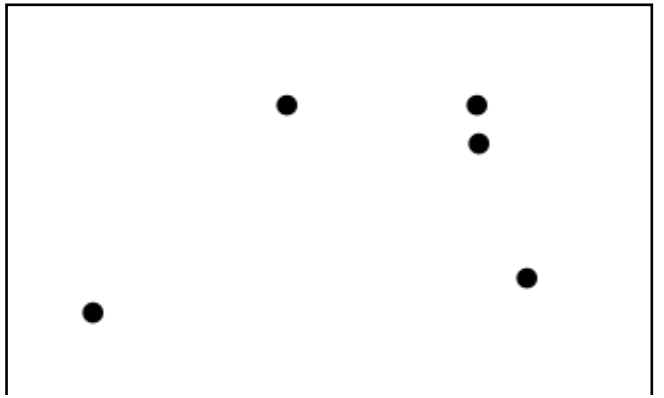
$f(r, v, t)$

6D+1

N-body – Discretization of gravity

Self-gravitating fluid:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + v\frac{\partial f}{\partial r} - \frac{\partial \Phi}{\partial r}\frac{\partial f}{\partial v} = 0 \qquad \nabla^2\Phi = 4\pi G \int f\,dv = 4\pi G\rho$$



Fluid is discretized into particles, which phase-position (r_a, v_a, t_a) represent the phase-space/mass distribution.

Accuracy depends on number of particles used to sample the underlying distribution (Monte-Carlo method).

Gravitational potential given by the integral of Poisson equation

$$f(\mathbf{r},\mathbf{v}) \rightarrow \sum_{i=1}^{N} m_i \delta^3(\mathbf{r}-\mathbf{r}_i)\delta^3(\mathbf{v}-\mathbf{v}_i)$$

$$(\dot{\mathbf{r}},\dot{\mathbf{v}}) = (\mathbf{v}, -\nabla\Phi)$$

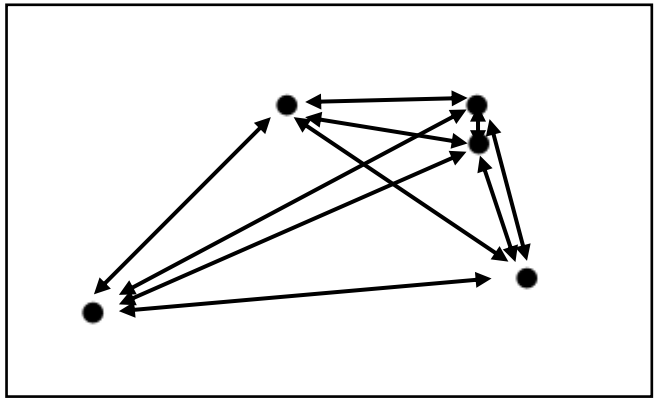
$$\Phi(r) = -G\sum_{b=1}^{N}\frac{m_b}{|r-r_b|}$$

Gravity solver

Challenge: global force need contribution from all

Naïve gravity implementation(O(N^2)):

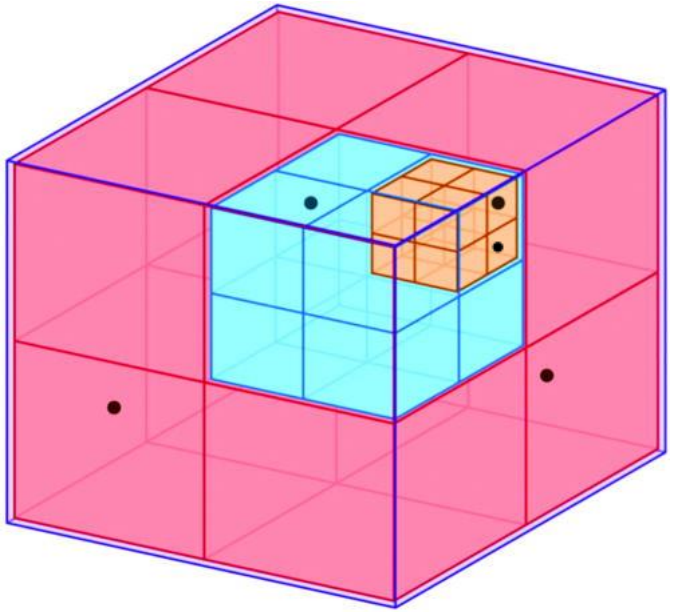
Each particle interacts directly with each other



$$\Phi(r) = -G \sum_{b=1}^{N} \frac{m_b}{|r - r_b|}$$

Gravity solver

Challenge: global force need contribution from all



Naïve gravity implementation(O(N^2)):

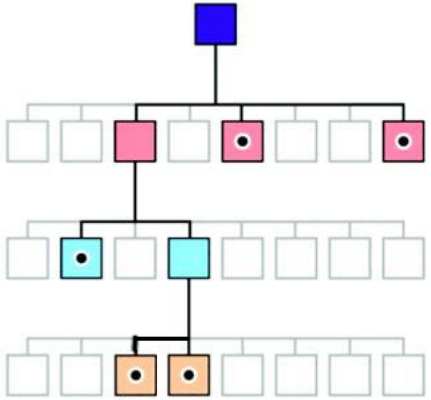
Each particle interacts directly with each other

Barnes-Hut approximation(O(NlogN)):

Separate force calculation to long-range and short-range

Influence from distant particles combined into center of mass (multipole expansion)

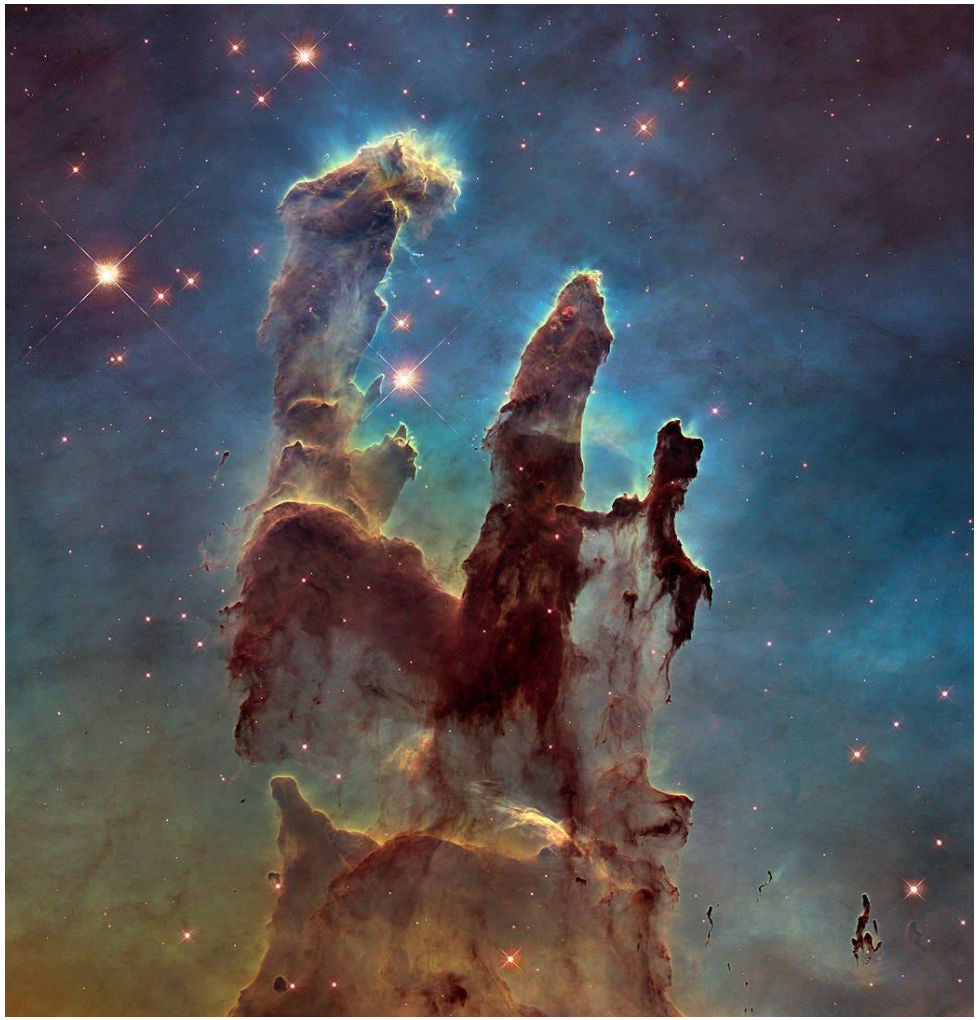
Tree traversal depends on a tolerance criterion



Dual-tree traversal (O(N))

In addition to particle-particle or particle-cell interactions, it also adds cell-cell interactions.

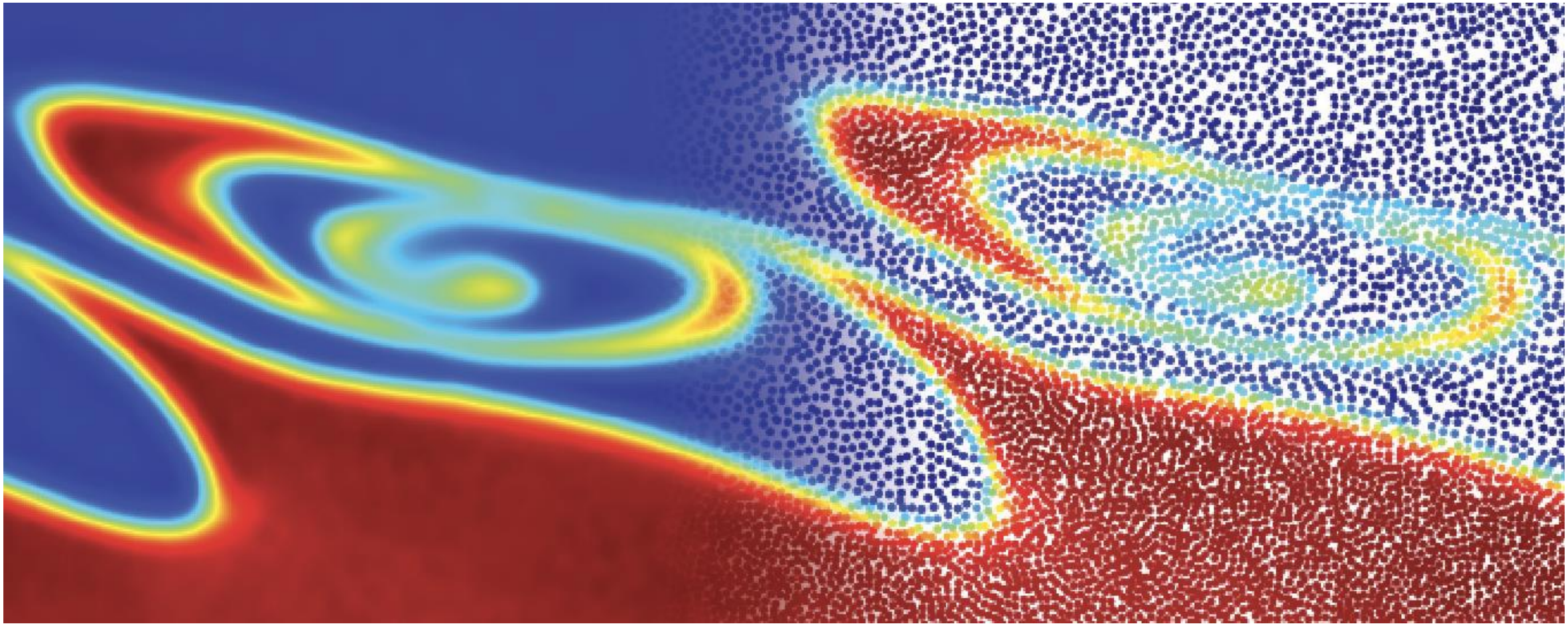
Hydrodynamics



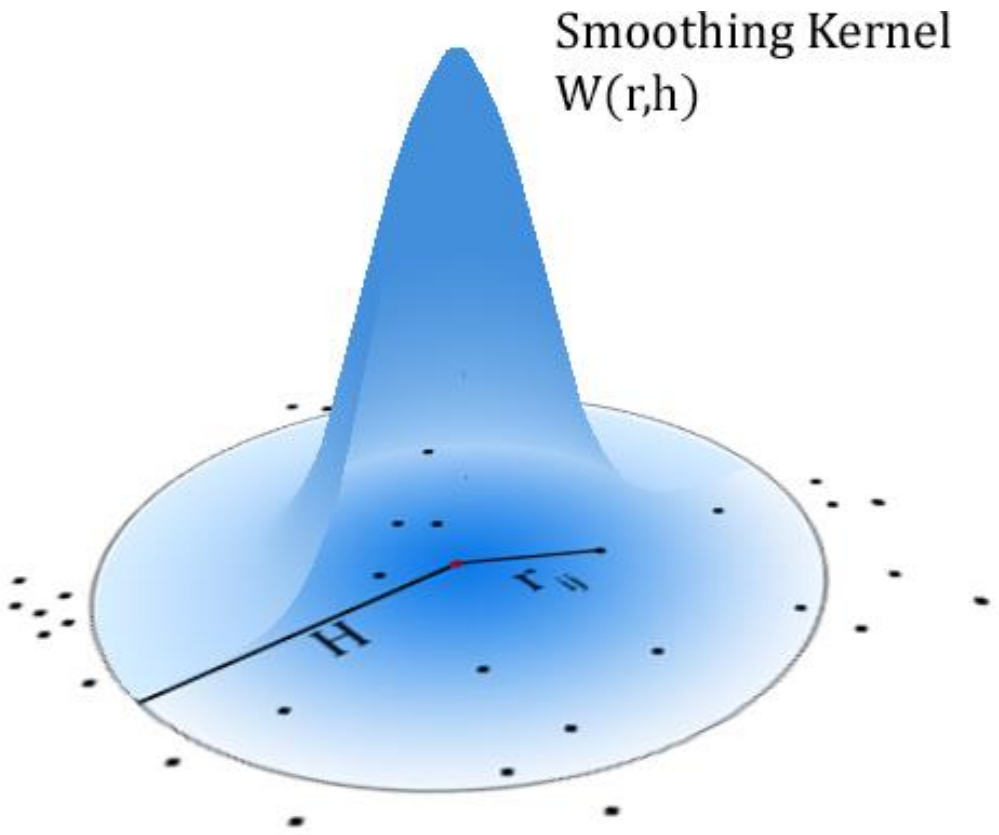




Smoothed particle hydrodynamics (SPH)



Smoothed particle hydrodynamics (SPH)



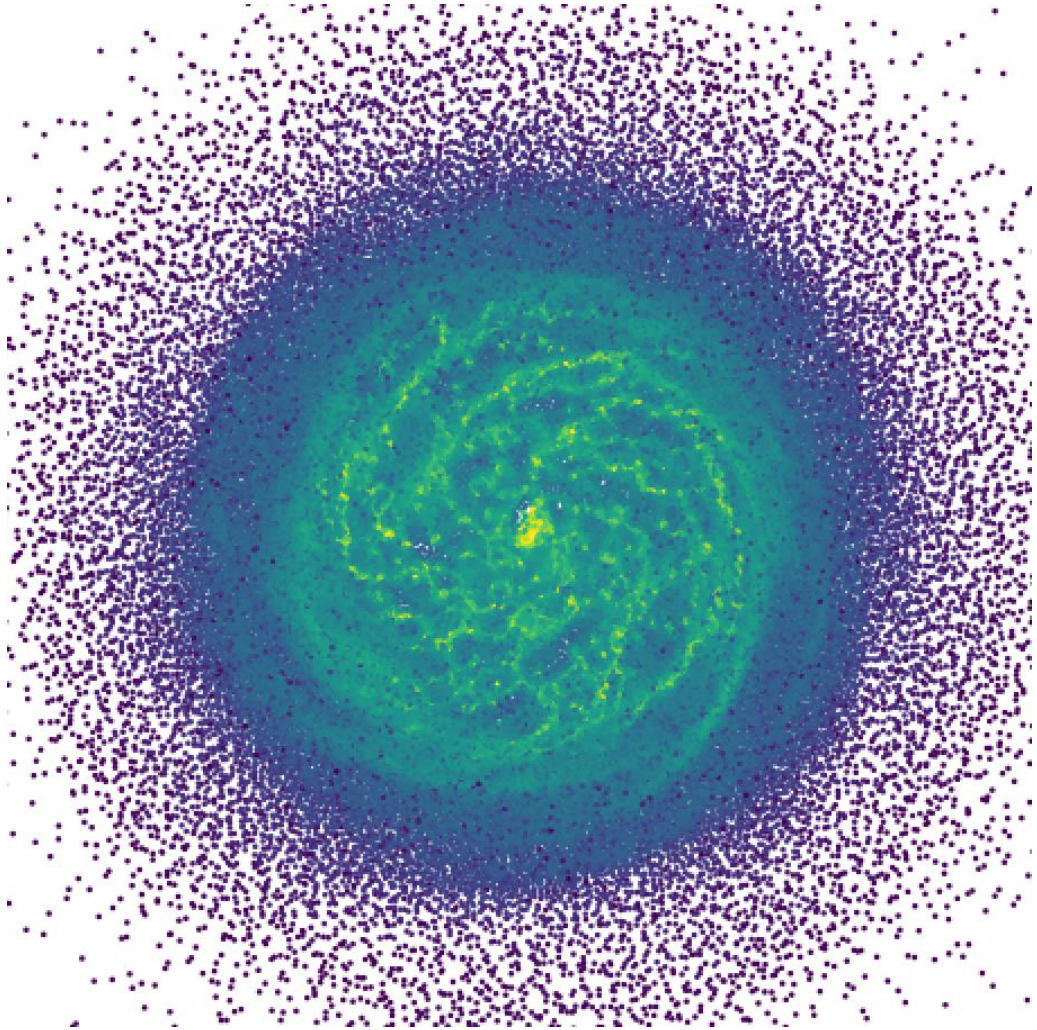
Smoothing length (h) represents our resolution scale. Similar to Δx in grid

- Lagrangian particle method
- Describes the continuous fluid at any point by interpolating over particles (interpolation points)
- Uses a **Kernel function** W for smoothing (Gaussian-like).
- Compact support/set number of neighbours

$$W(r,h) = 0 \; when \; r > H$$

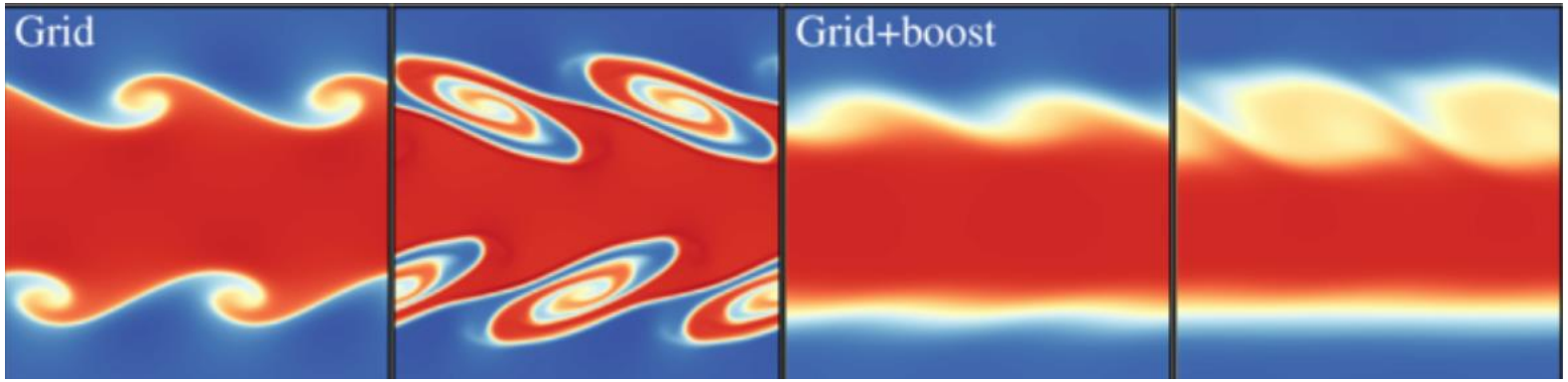
Advantages of SPH

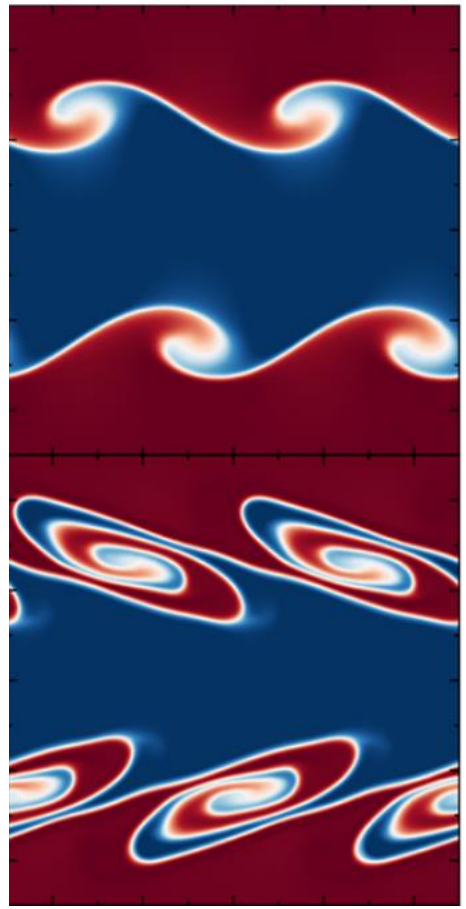
- Resolution follows mass



Advantages of SPH

- Resolution follows mass
- No intrinsic dissipation
- Galilean invariant





SPH

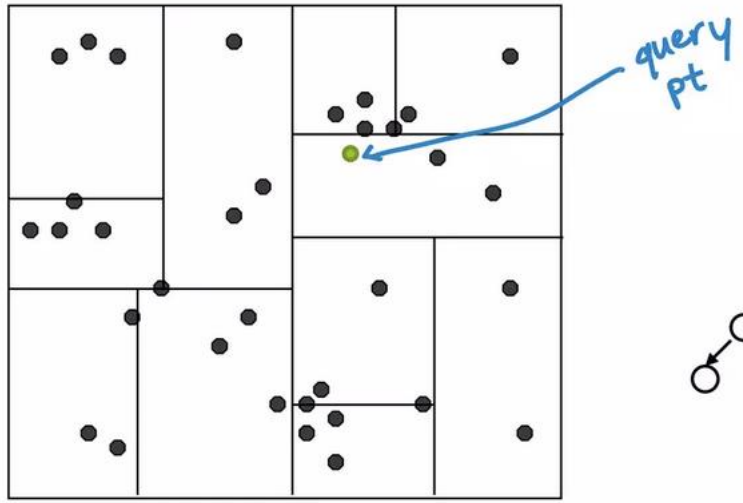
Advection/truncation errors in Eulerian codes are not Galilean invariant and increases with an imposed bulk velocity

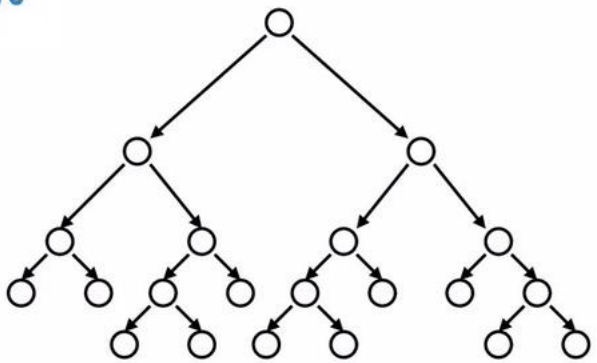
Advantages of SPH

- Resolution follows mass
- No intrinsic dissipation
- Galilean invariant
- Exact solution to the continuity equation
- Excellent conservation of momentum, angular momentum, energy and entropy
- Easy tracking of material evolution
- Couples directly with N-body codes (gravity)

K-Nearest neighbour

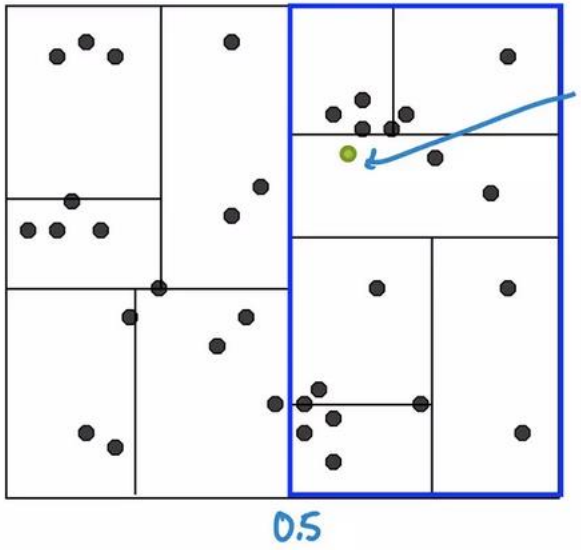
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**

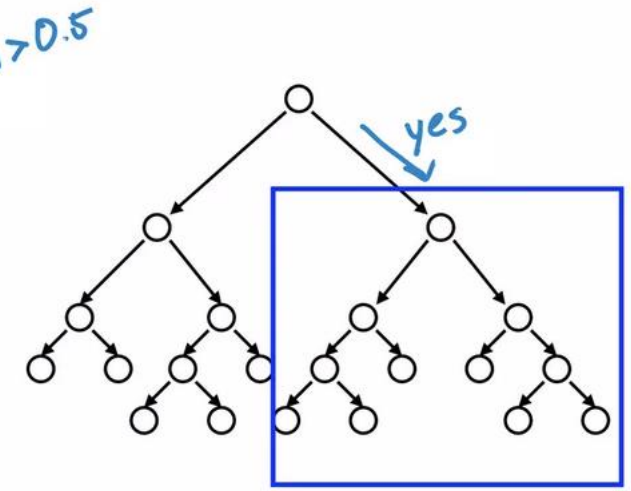




K-Nearest neighbour

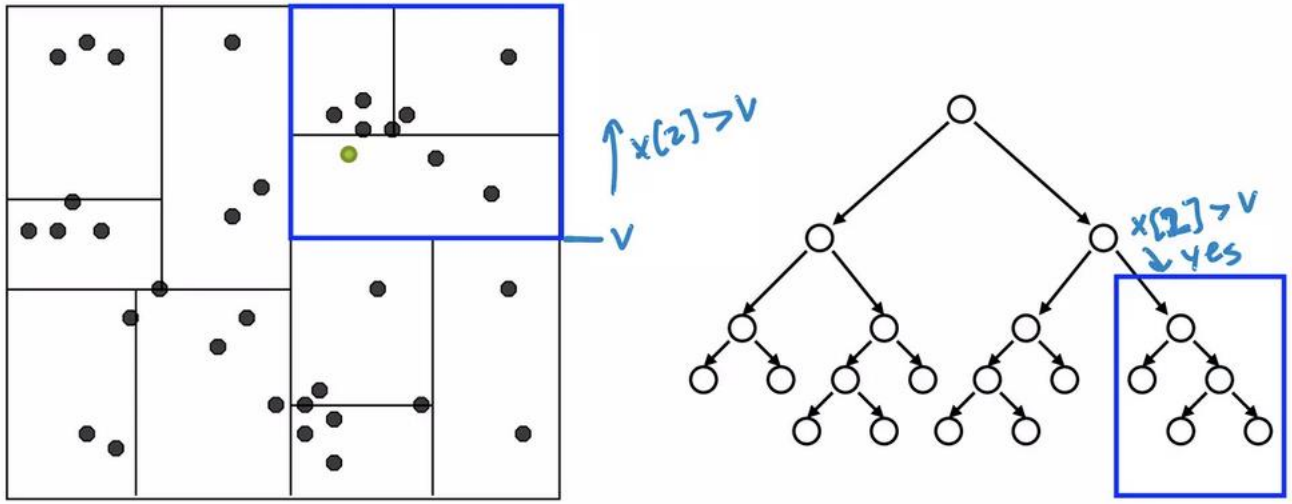
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**





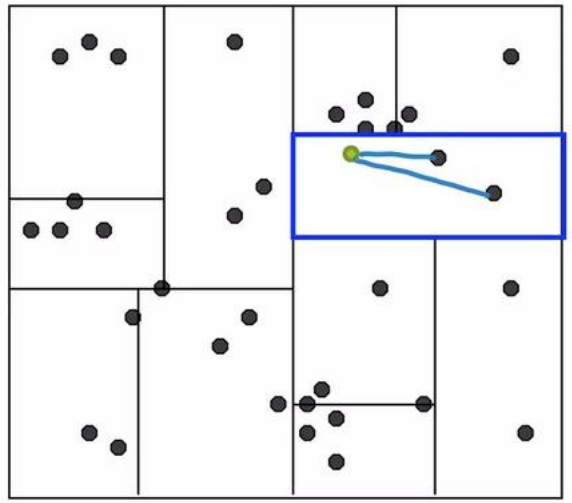
K-Nearest neighbour

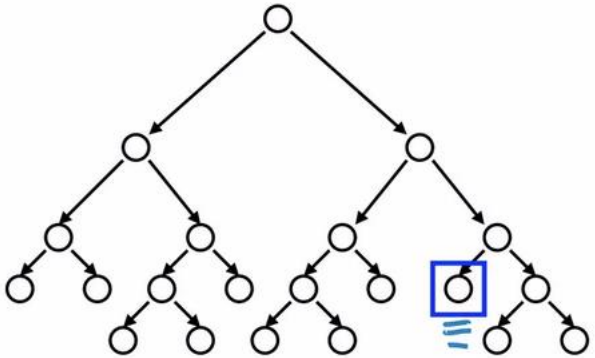
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**



K-Nearest neighbour

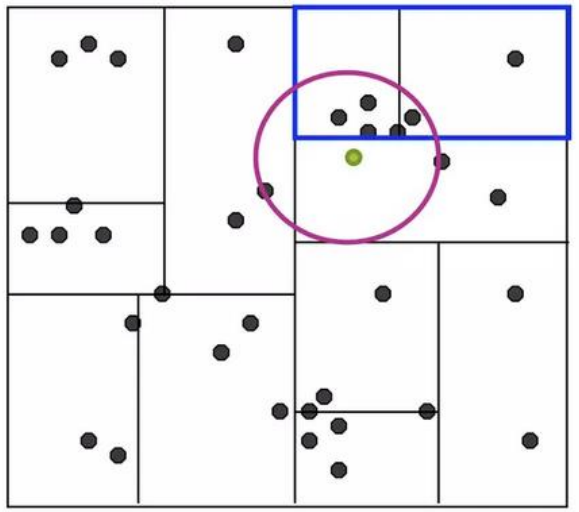
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**

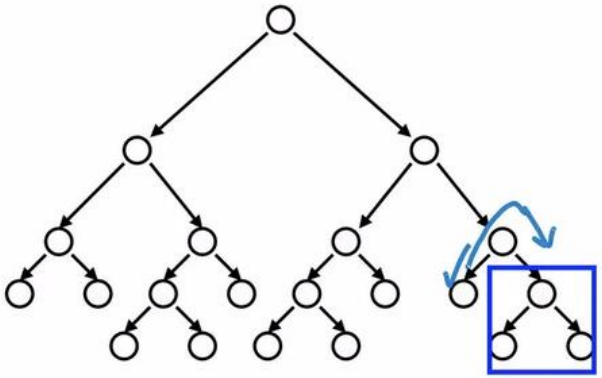




K-Nearest neighbour

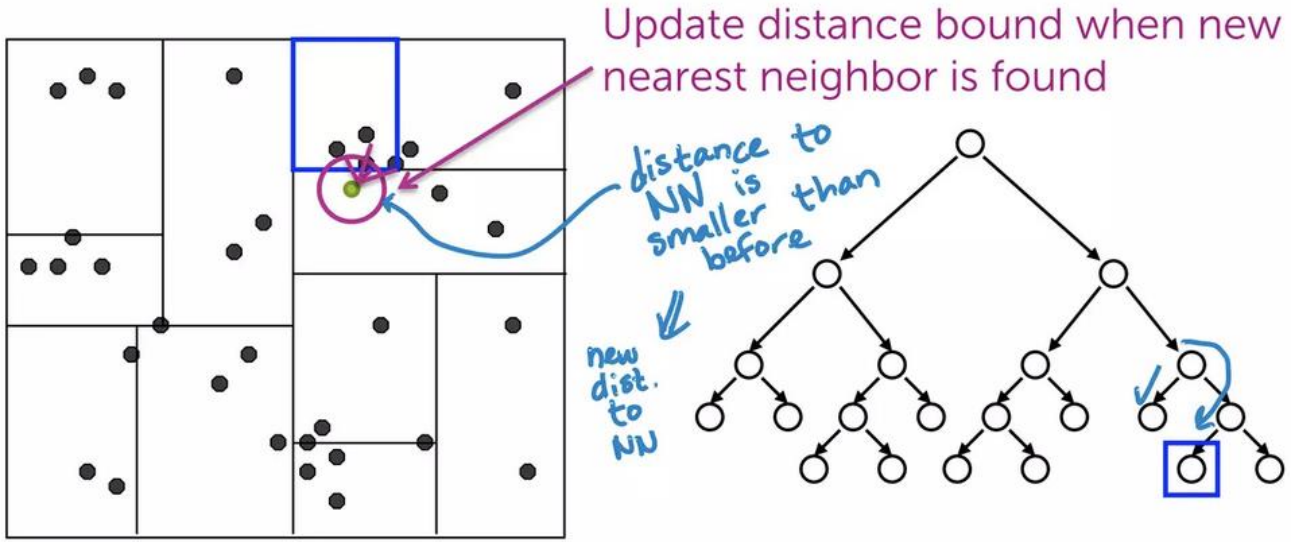
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**





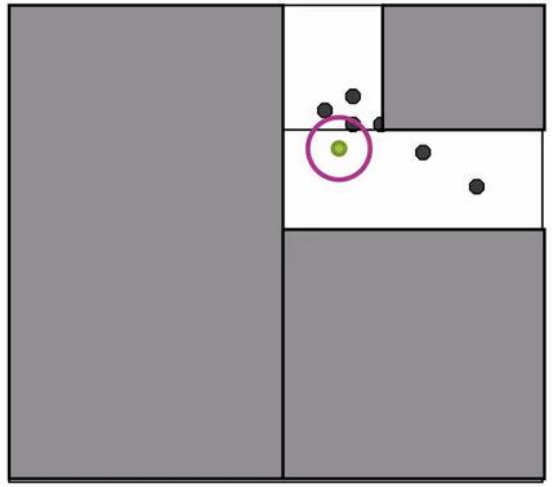
K-Nearest neighbour

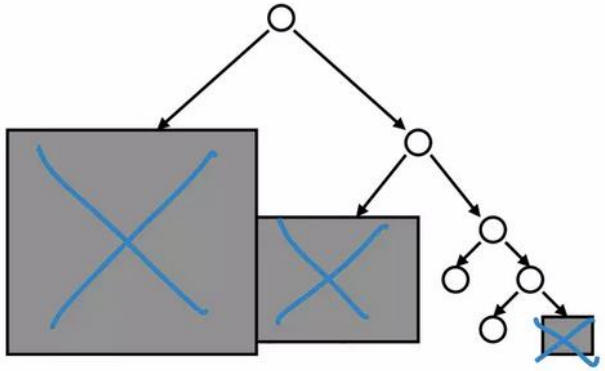
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**



K-Nearest neighbour

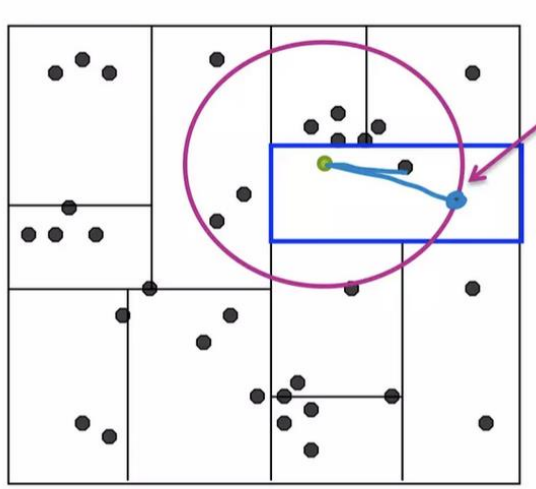
- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**

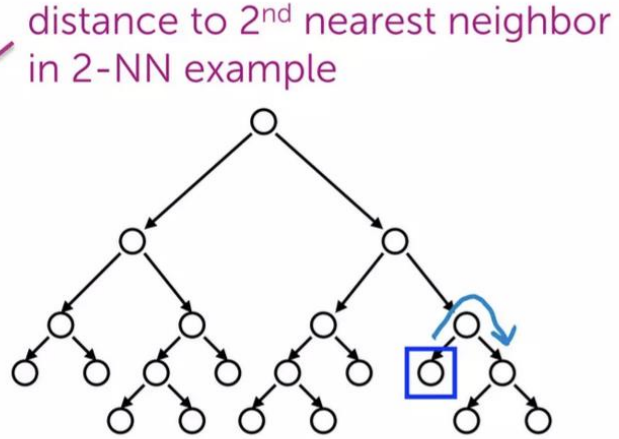




K-Nearest neighbour

- Goal: Find the k nearest neighbours
- Brute force method -> calculate distance to each point
- Combine with tree build to reduce search area **O(NlogN)**
- k usually around 64





Exactly same algorithm, but maintain distance to furthest of current *k* nearest neighbors

Remaining major physics algorithms

Feedback

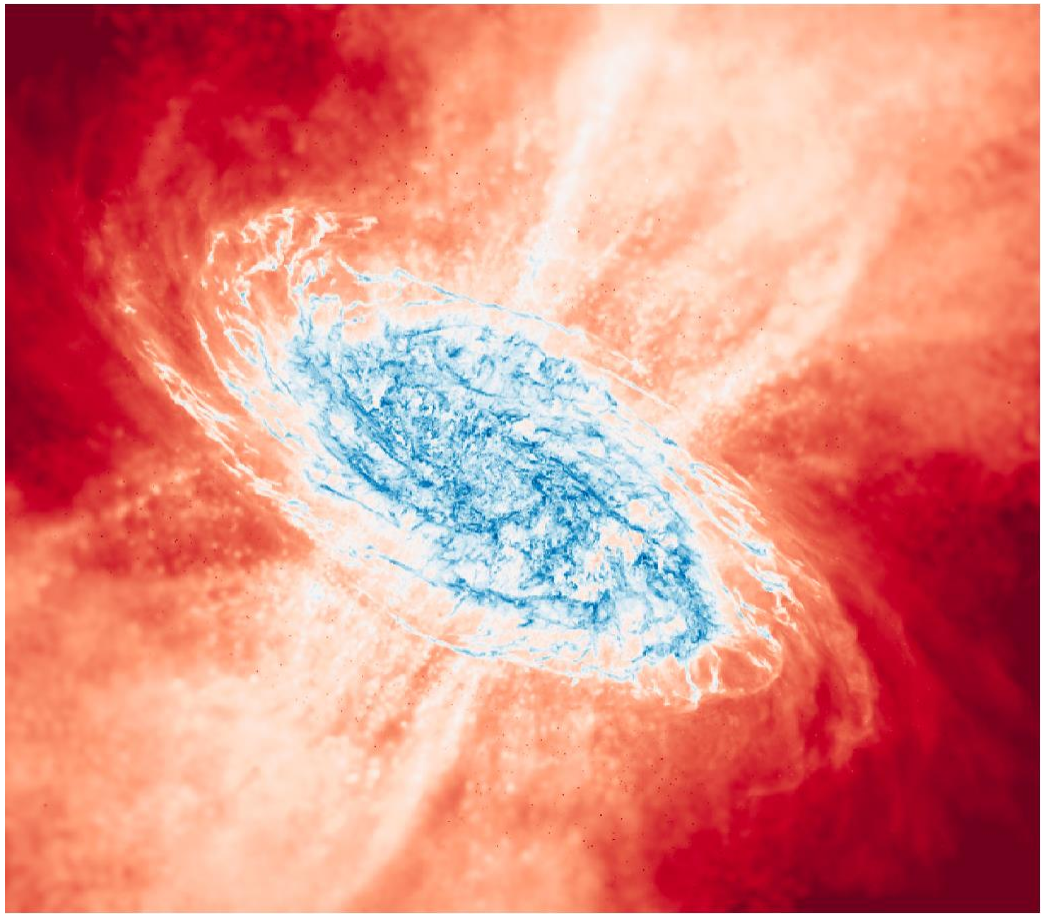
Similar computational challenges as SPH, using k-nearest neighbours to distribute feedback energy, involves both star and gas particles.

Radiative cooling

Semi-implicit module
Large integration times needed
when cooling timestep is << dynamical timestep

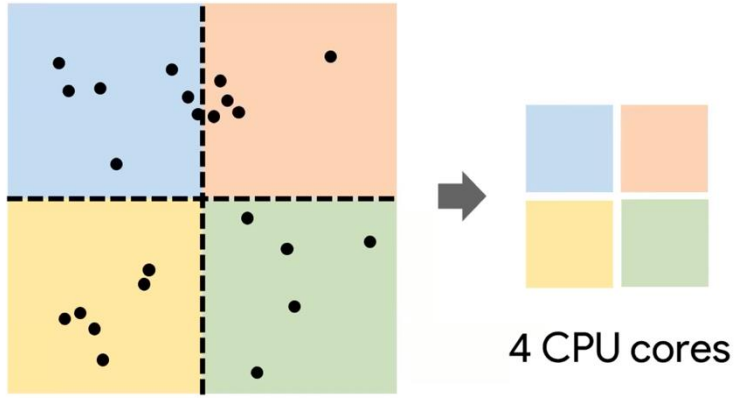
Cosmological boundaries

Ewald summation used to speed up convergence of gravity contribution from the rest of universe



Domain decomposition

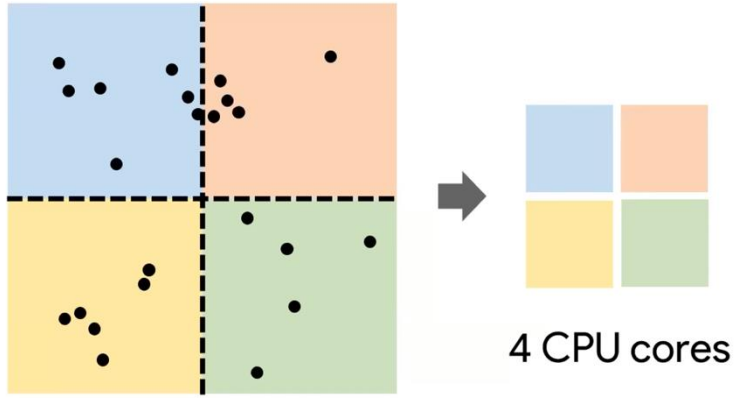
- How do we best split up and distribute work among processors
- Description of elements (particles) as 1D “keys”



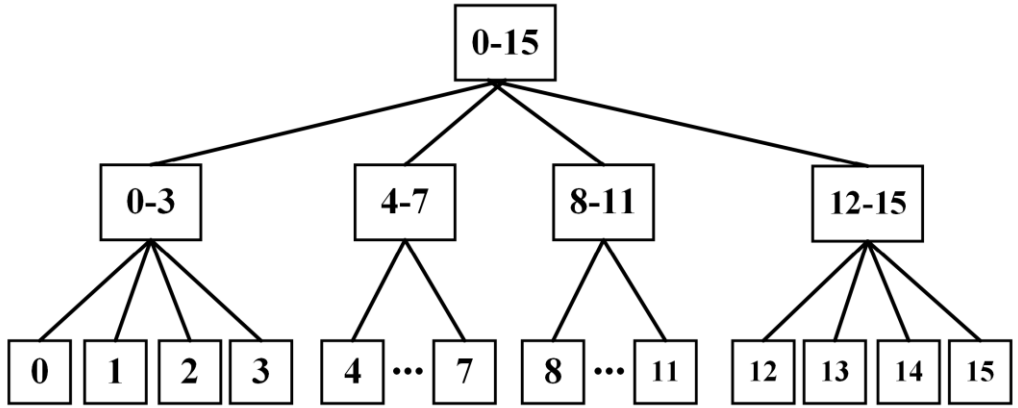
Per-process decomposition
(MPI)

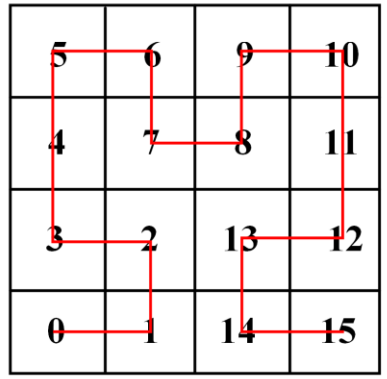
Domain decomposition

- How do we best split up and distribute work among processors
- Description of elements (particles) as 1D “keys”
- Ordering these keys depending on decomposition(Morton, Peano-Hilbert, etc.)
- Splitting this list of keys (work objects) and distributing among processors



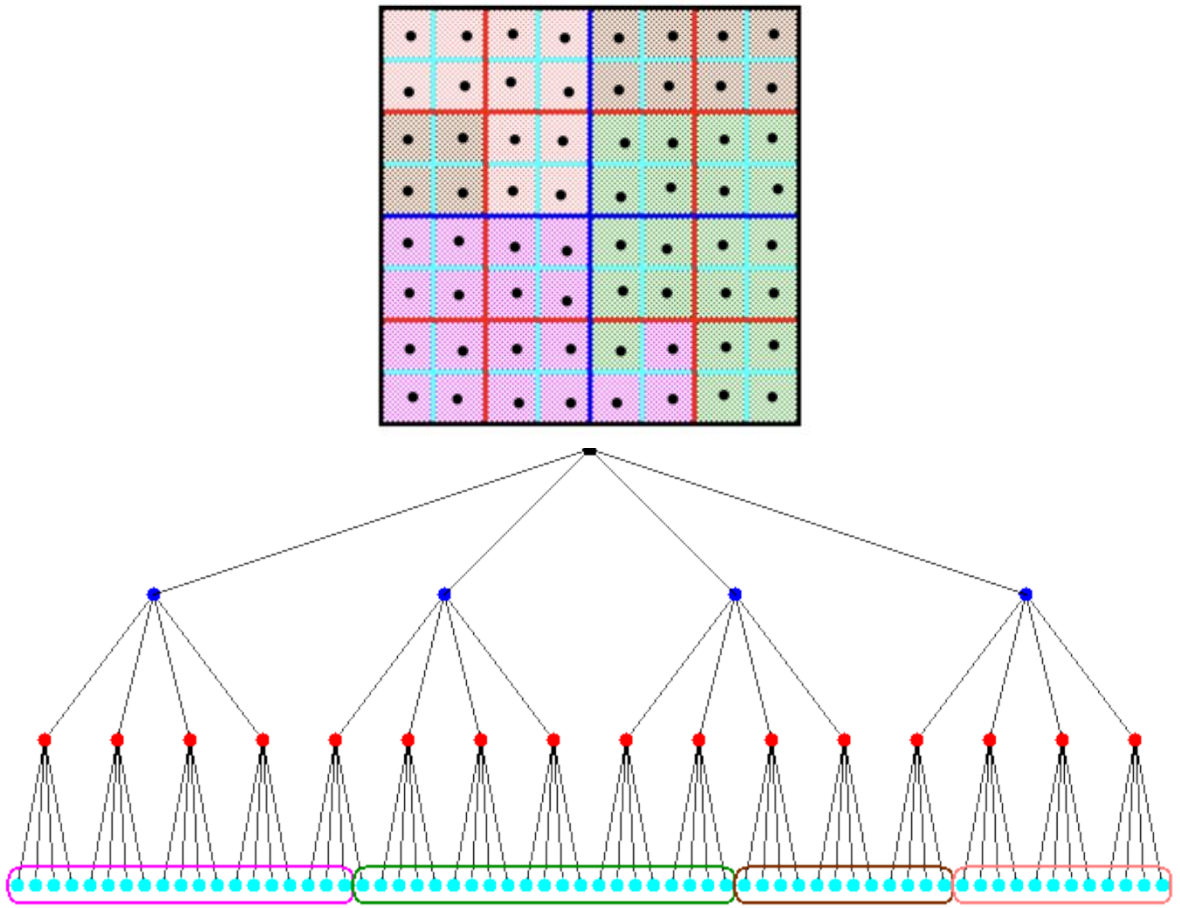
Per-process decomposition (MPI)





Domain decomposition

- How do we best split up and distribute work among processors
- Description of elements (particles) as 1D “keys”
- Ordering these keys depending on decomposition(Morton, Peano-Hilbert, etc.)
- Splitting this list of keys (work objects) and distributing among processors
- Load balancing -> changing the number of elements within each work object to balance load

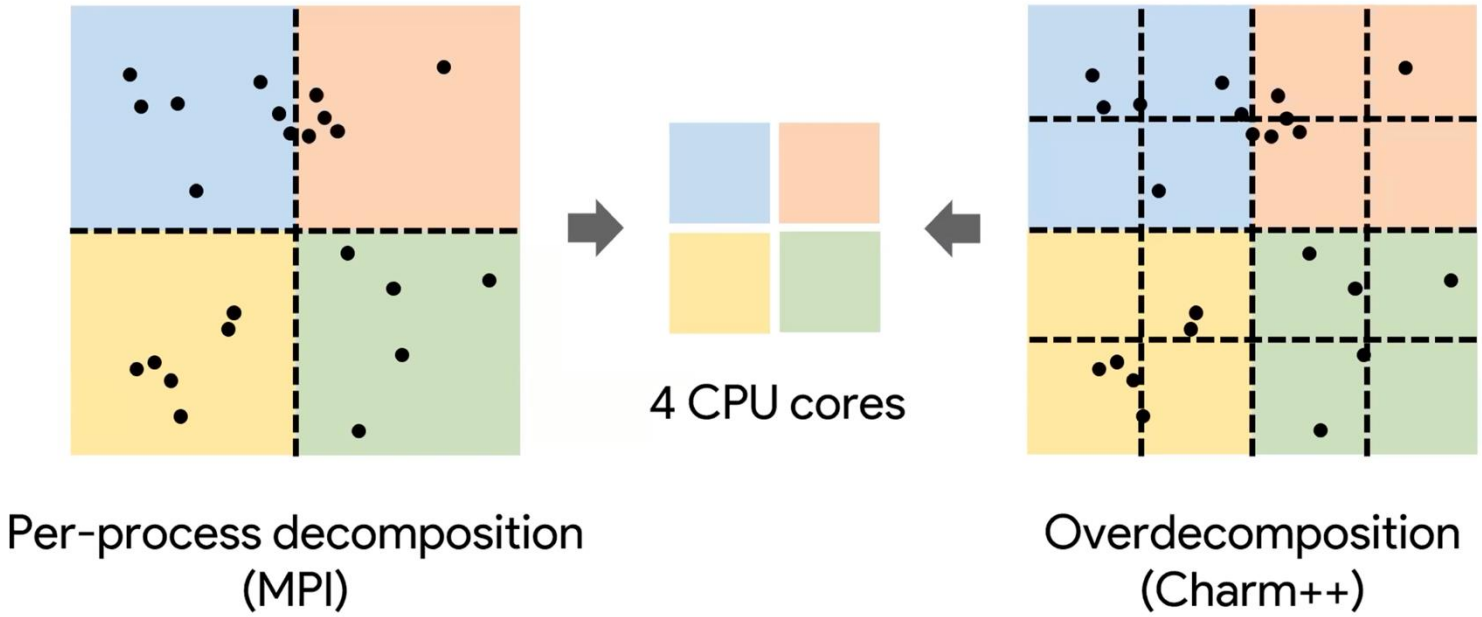


Demmel CS267 lecture

ChaNGa – Charm++

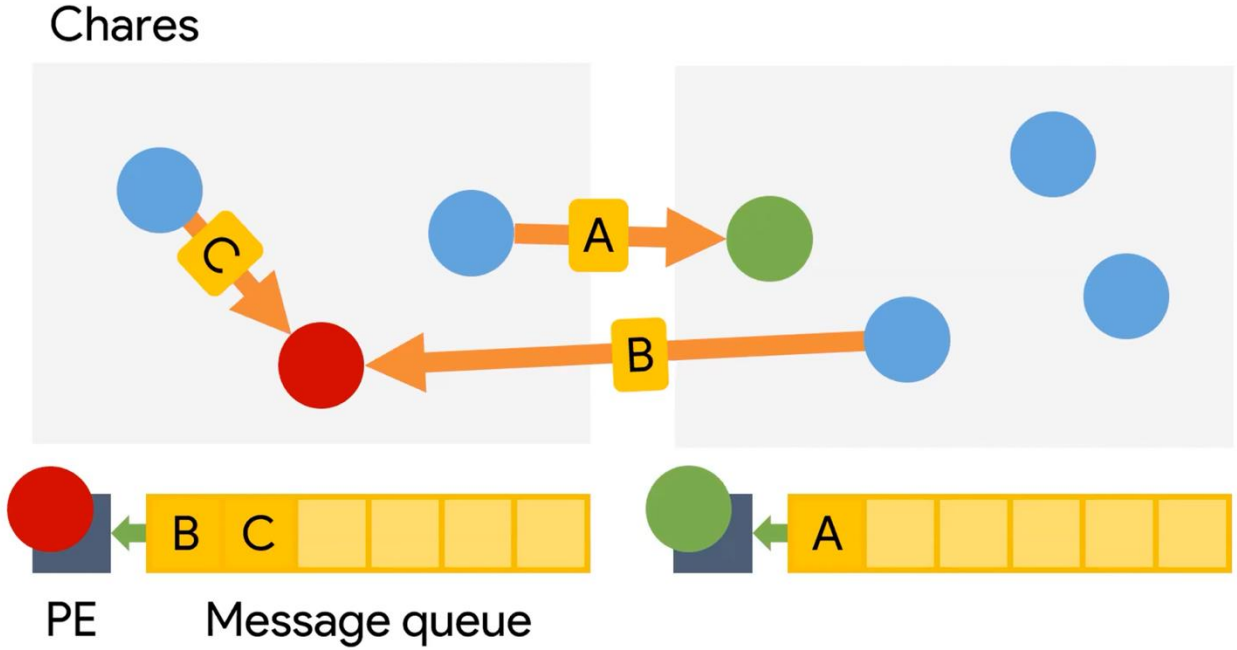
- Charm++ runtime (PPL, UIUC)
  C ++ - based parallel runtime system
- Allows for over-decomposition, more work objects (chares) than there are processors

- These objects are migrated to other processors during runtime to improve load balancing with little user intervention



ChaNGa – Charm++

- Charm++ runtime (PPL, UIUC)
  C ++ - based parallel runtime system
- Allows for over-decomposition, more work objects (chares) than there are processors

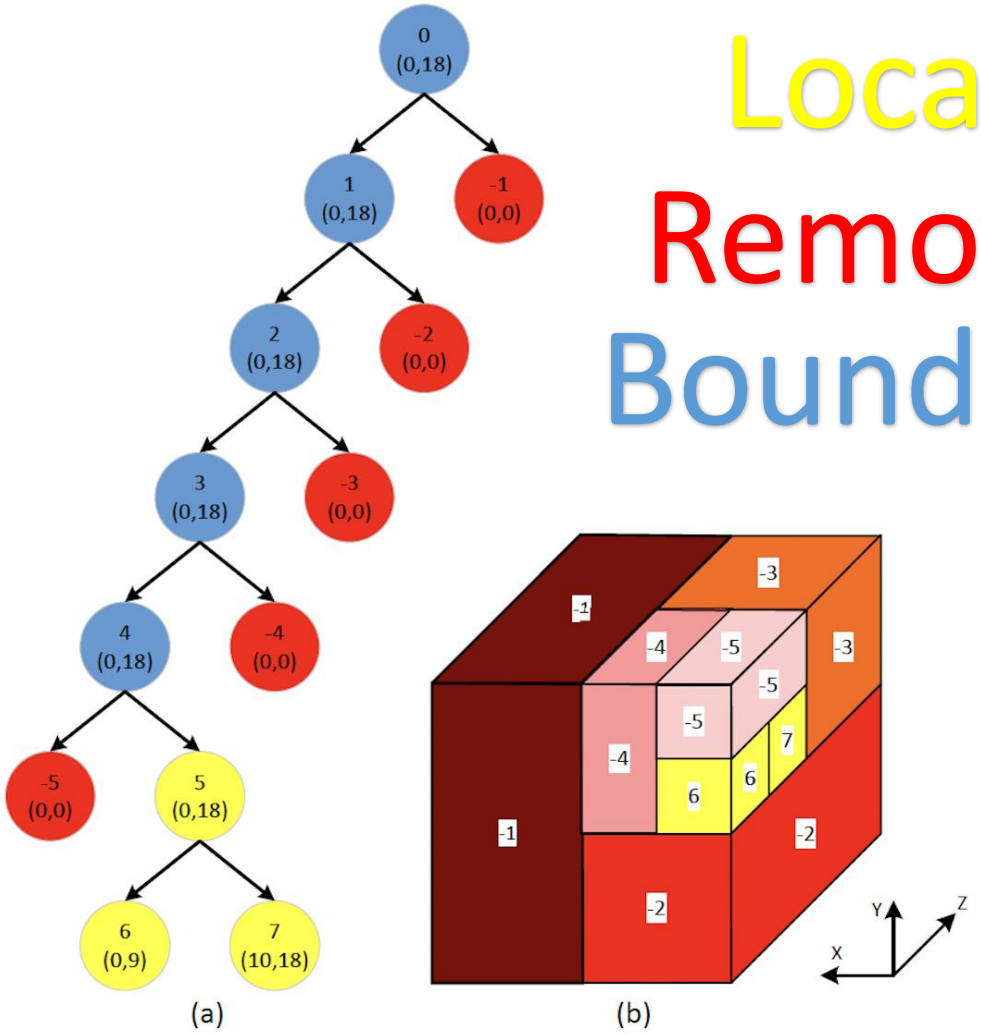


- These objects are migrated to other processors during runtime to improve load balancing with little user intervention

- Chares calls entry methods on each other (return void+immediate+ message queued)
- Computation and communication can be overlapped effectively

ChaNGa – TreePieces

- The work objects (“chares”) in ChaNGa are TreePieces
- Global tree split into N number of TreePieces (depending on over-decomposition)
- Leaves are buckets of particles
- Tree walks/computation separated into local and remote



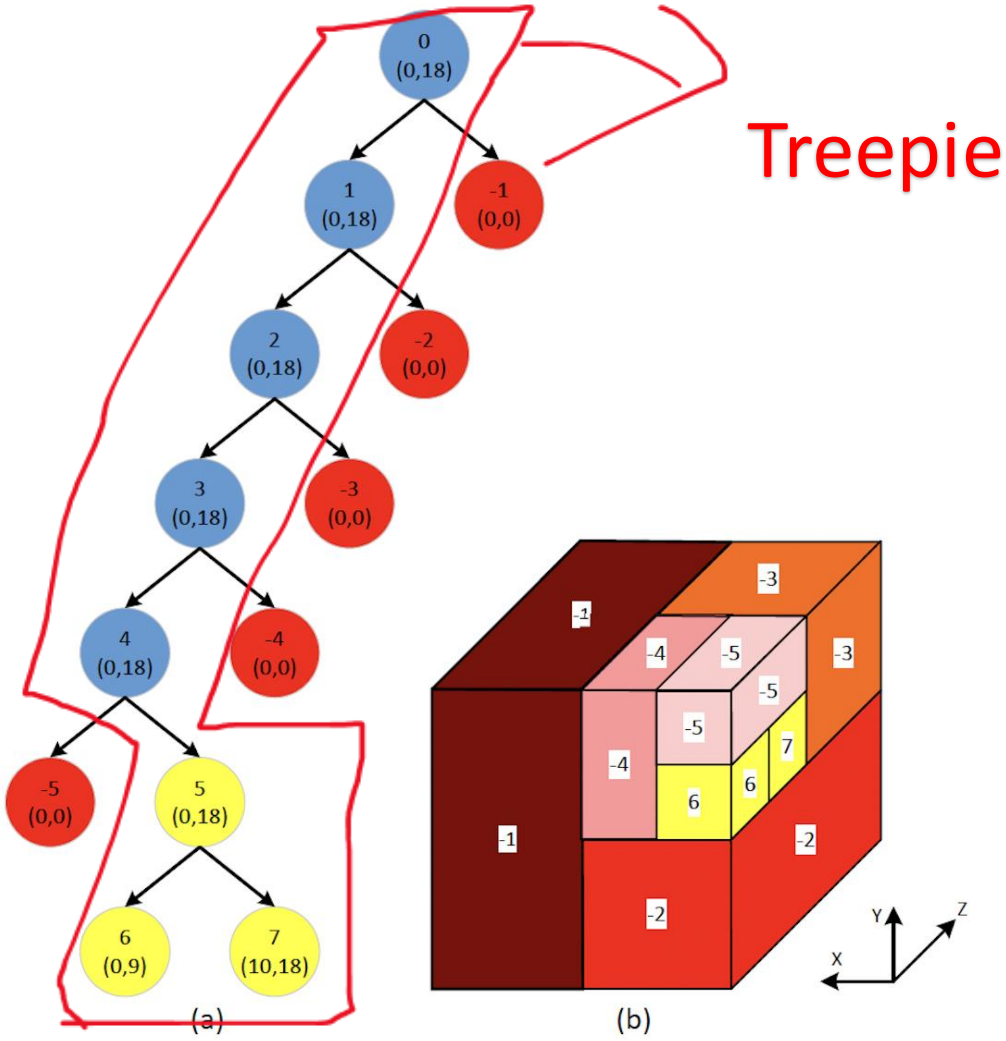
Local

Remote

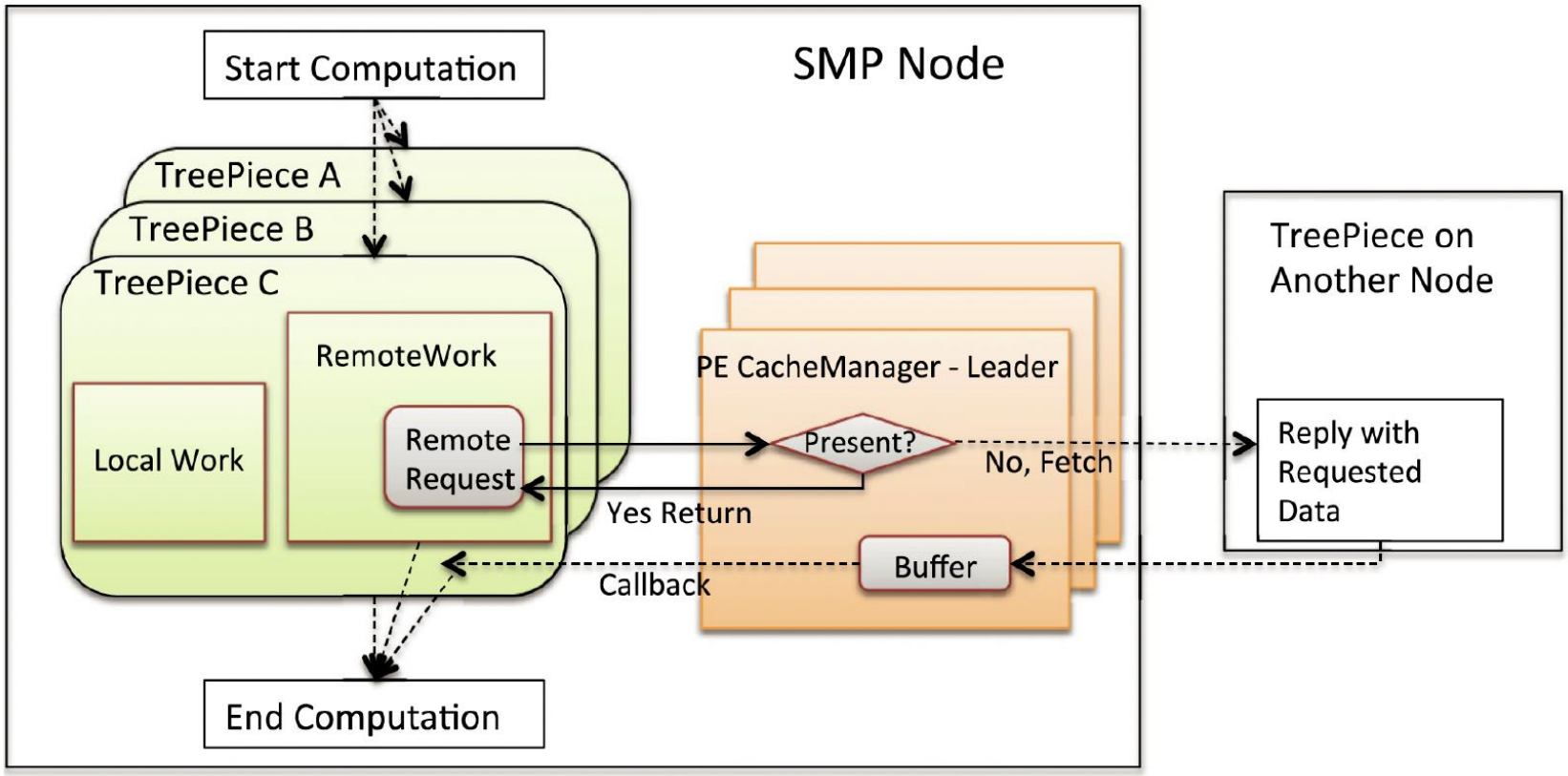
Boundary

ChaNGa – TreePieces

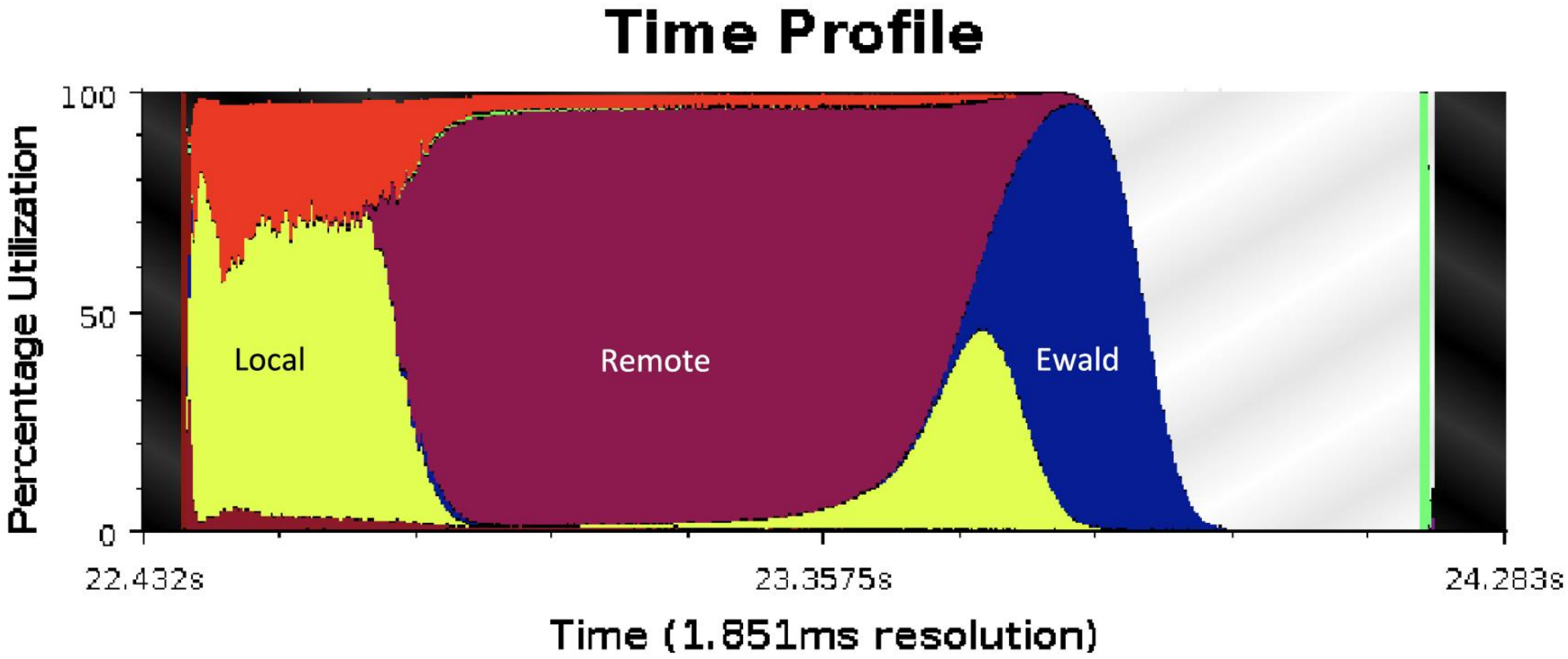
- The work objects (“chares”) in ChaNGa are TreePieces
- Global tree split into N number of TreePieces (depending on over-decomposition)
- Leaves are buckets of particles
- Tree walks/computation separated into local and remote



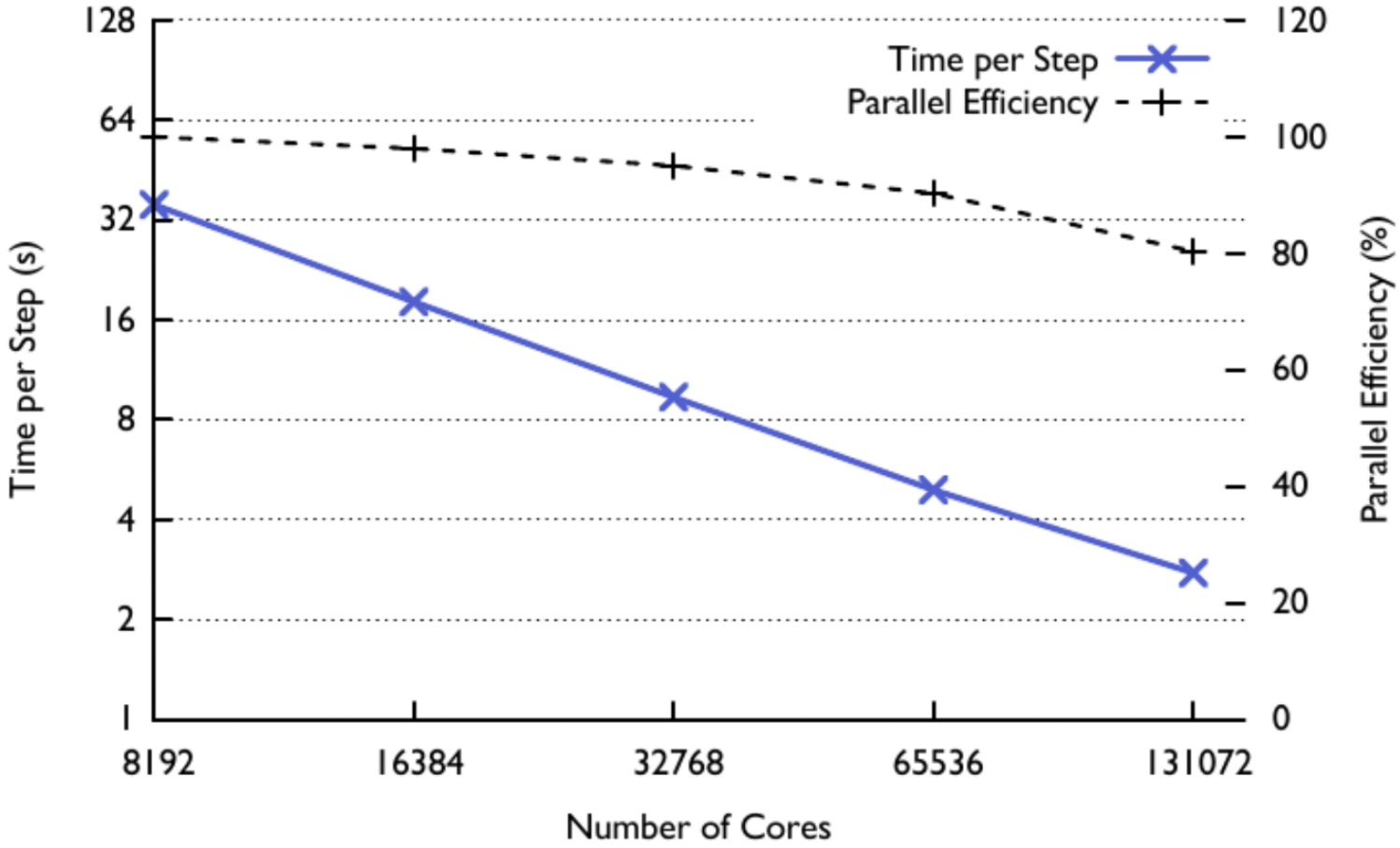
Tree walk


Start Computation
SMP Node
TreePiece A
TreePiece B
TreePiece C
RemoteWork
Local Work
Remote Request
PE CacheManager - Leader
Present?
No, Fetch
Yes Return
Buffer
Callback
TreePiece on Another Node
Reply with Requested Data
End Computation

ChaNGa – Charm++

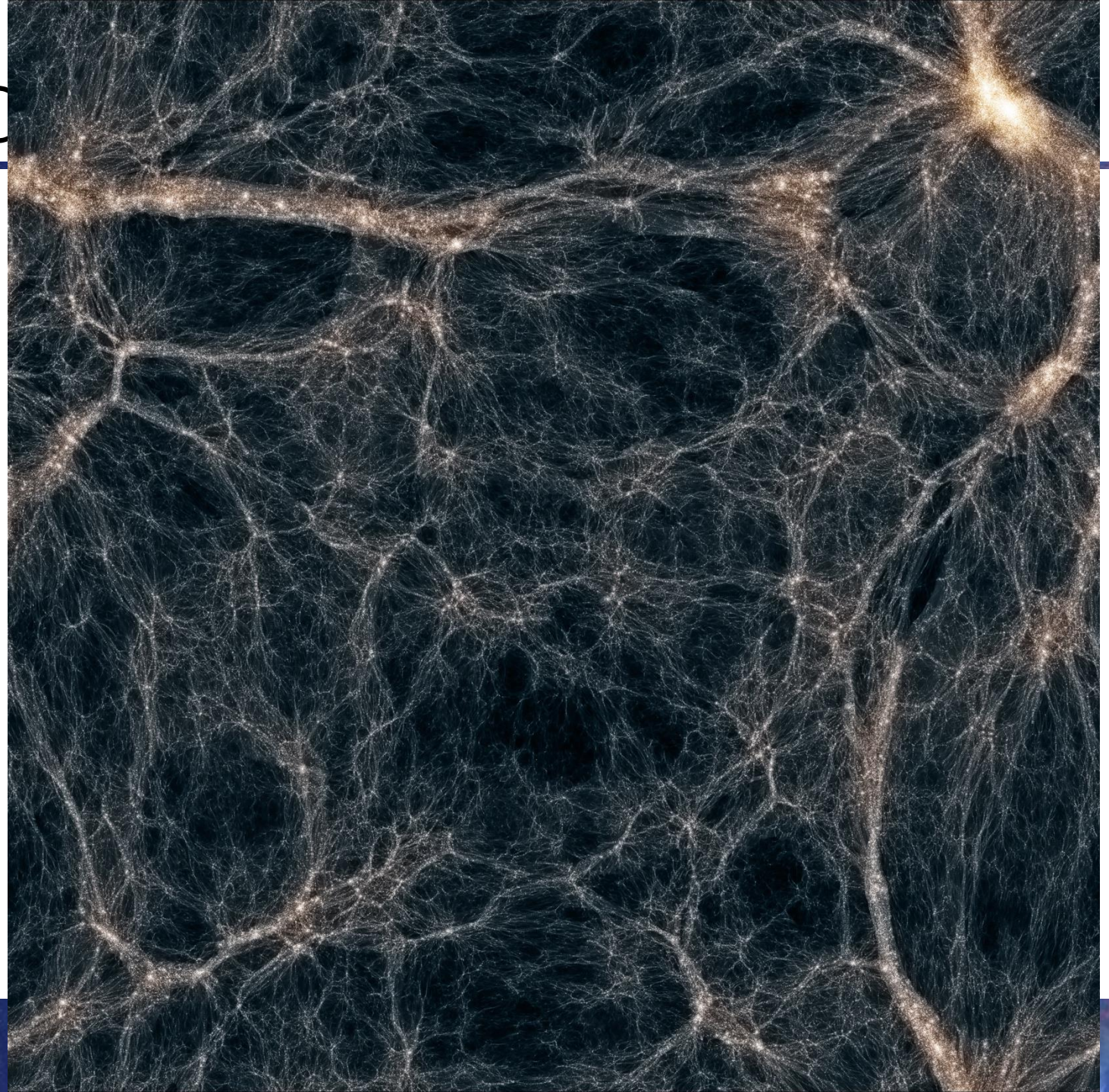


ChaNGa CPU Scaling: Gravity Only



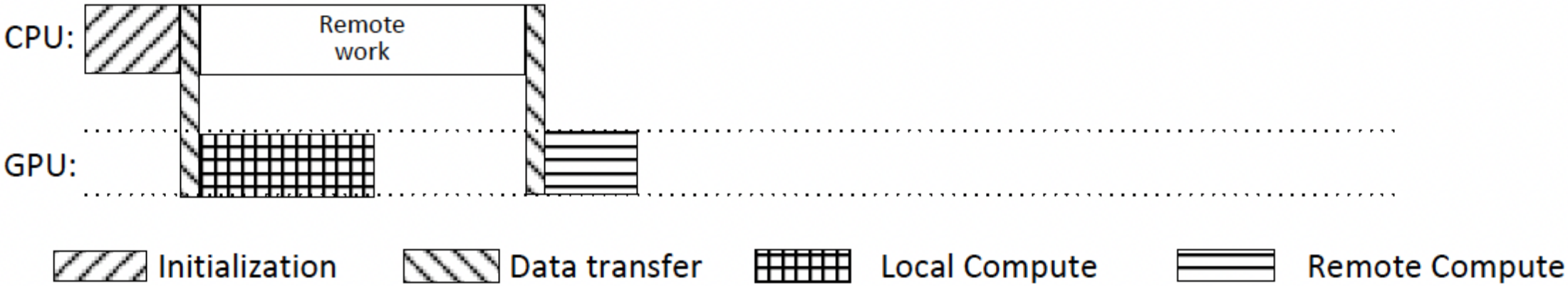
Cosmological simulation; 2 billion particles; Gravity only

ChaNGa C



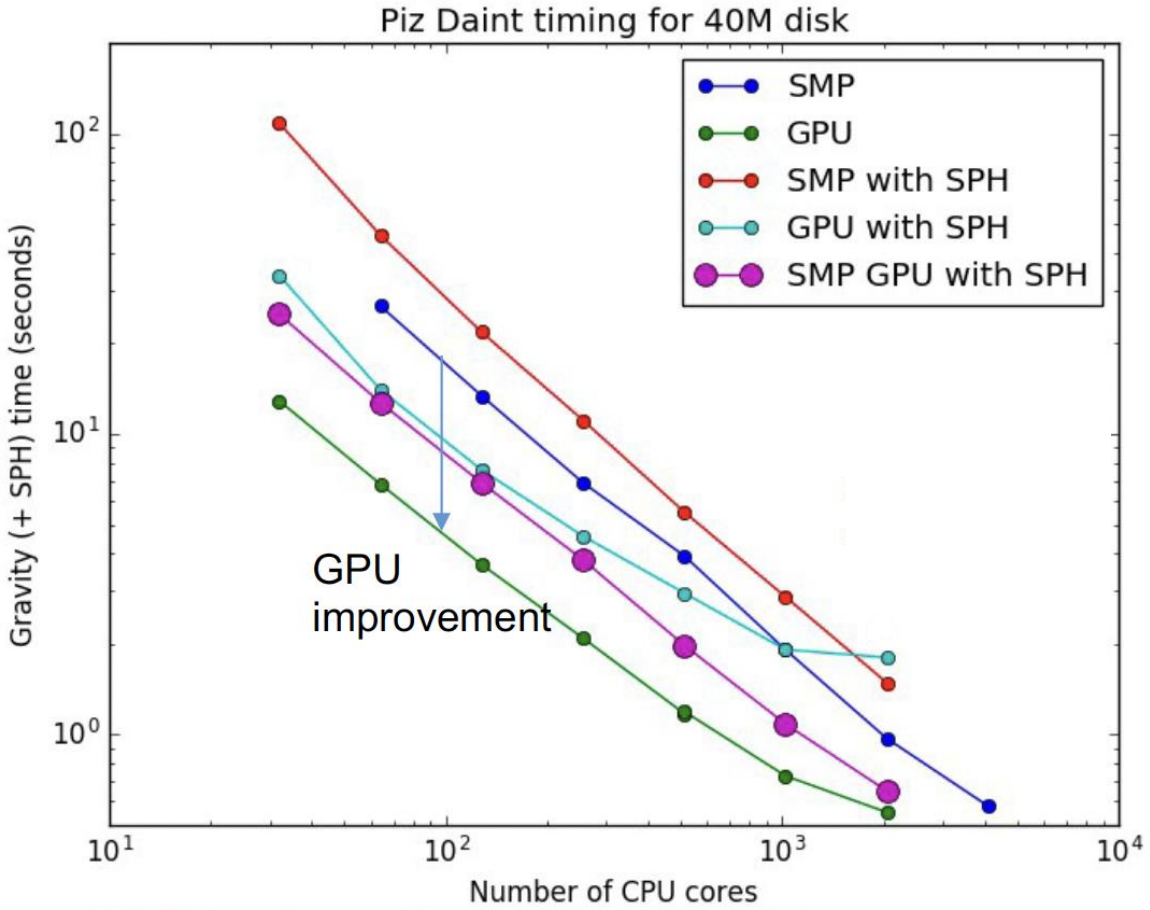
GPU offloading for gravity

The GPU handles both the local tree walk and force computation (single-tree walk for improved parallelism).

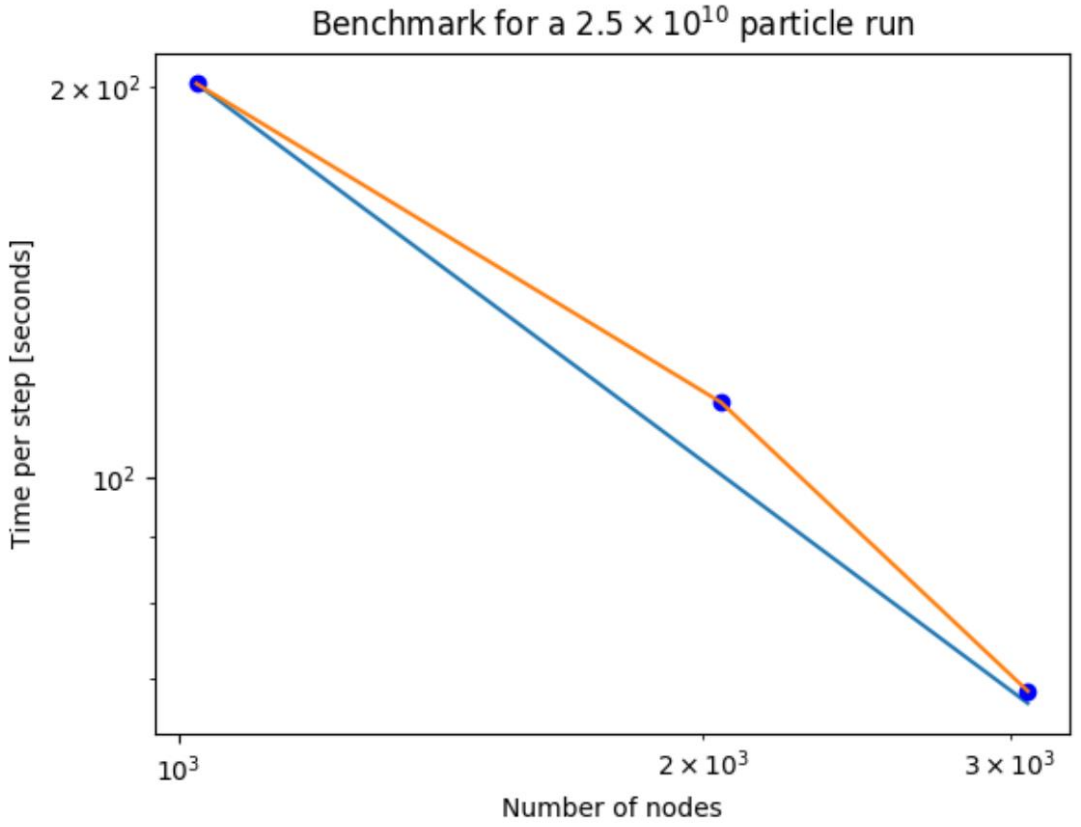


Liu et al. 2019

GPU offloading for gravity



GPU implementation (gravity only);





Future Plans and Recent Progress

Co-funded by the European Union

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Belgium, Czech Republic, France, Germany, Greece, Italy, Norway, and Spain under grant agreement No 101093441.

EuroHPC Joint Undertaking

What is SPACE ?

Stands for scalable parallel astrophysical codes for exascale

Newly funded Centre of Excellence (CoE), which aims to prepare astrophysics applications for the exa-scale and beyond.

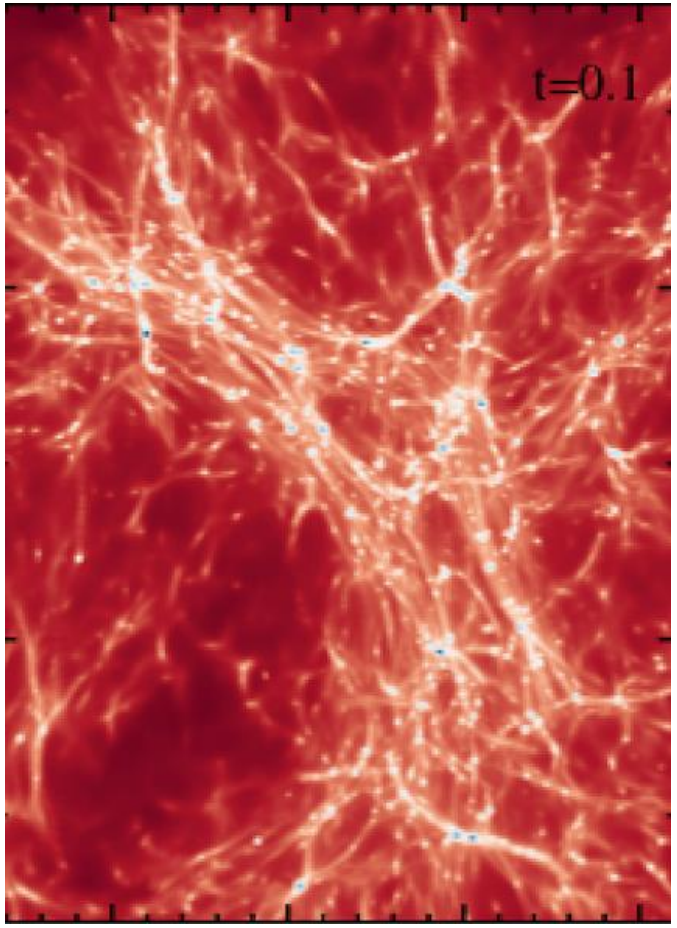
Funded by the Eurohpc joint undertaking.

Brings together expertise from scientists, code developers, software developers and hpc experts



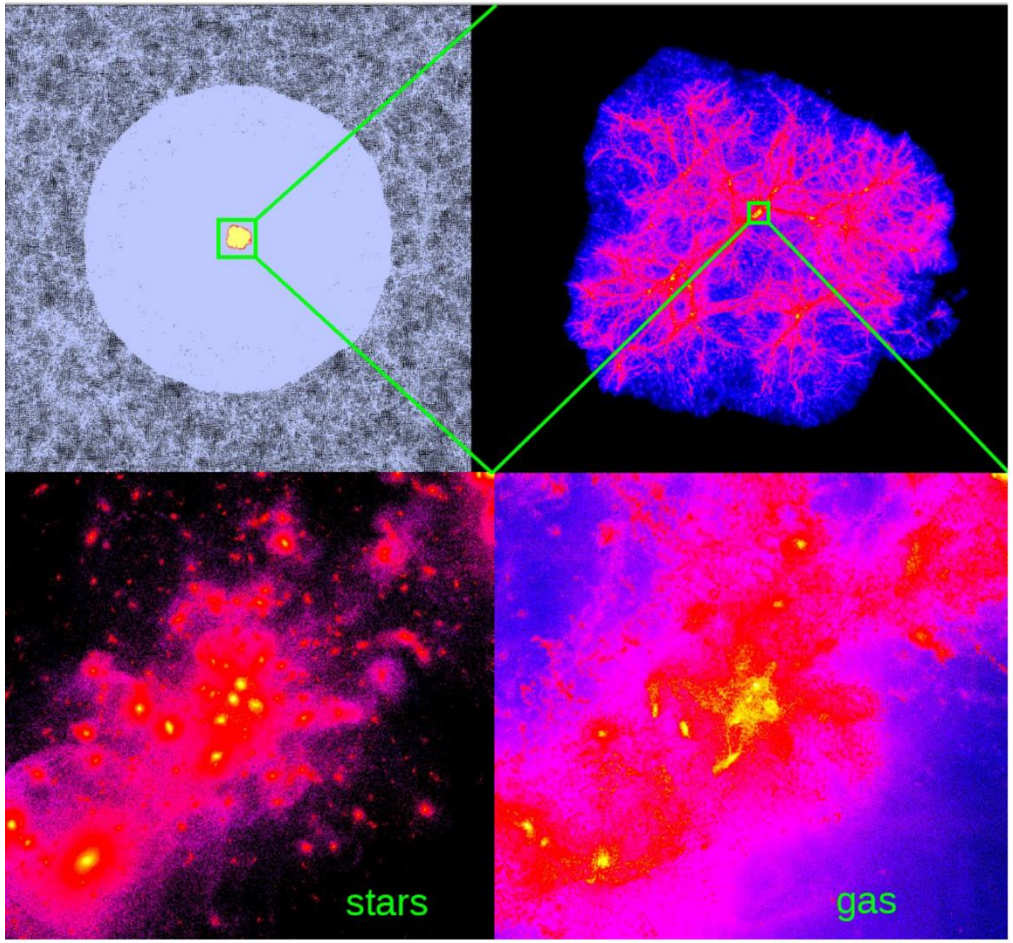
Science Case: Cosmological Volumes

- Simulation a patch of the Universe, capture large-scale structure and statistical properties of galaxies
- **Modules:** all major physics algorithms
- **Challenges:** limited resolution of individual galaxies, large dynamic range (6 orders of magnitude), huge data set for analysis/visualization
- **Why exascale?** To reach the resolution needed to model galaxy formation physics with more realistic subgrid recipes



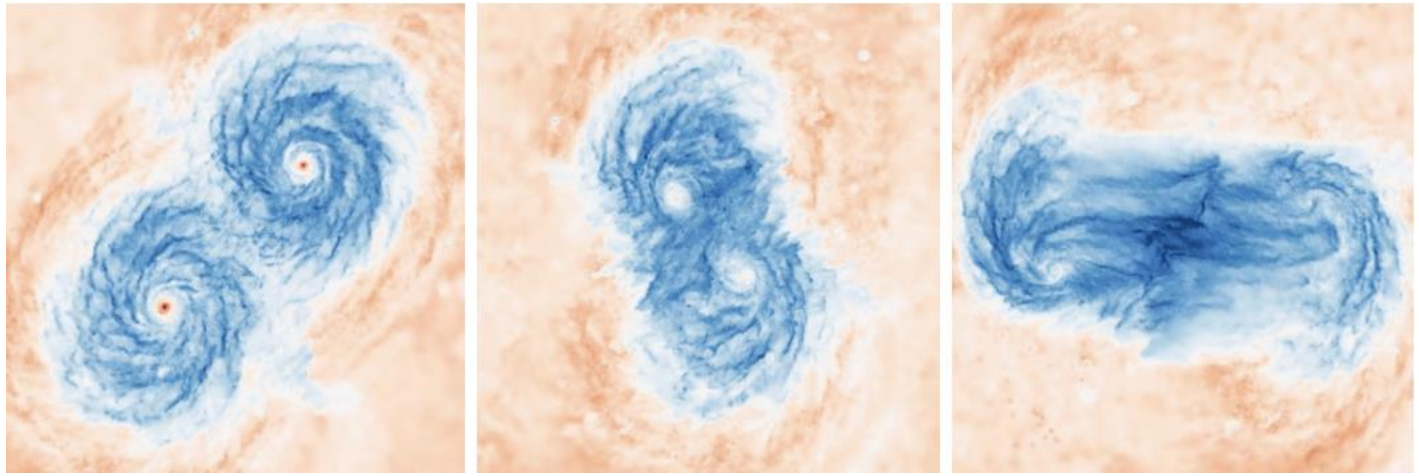
Science Case: Zoom-in Galaxy Simulations

- Simulation formation of individual galaxies with much higher resolution and more physical details while maintaining cosmological boundary
- **Modules:** all major physics algorithms
- **Challenges:** very clustered data, huge dynamic range (8-9 orders of magnitude), huge data set for analysis/visualization
- **Why exascale?** To reach the resolution needed to model galaxy formation physics with more realistic subgrid recipes



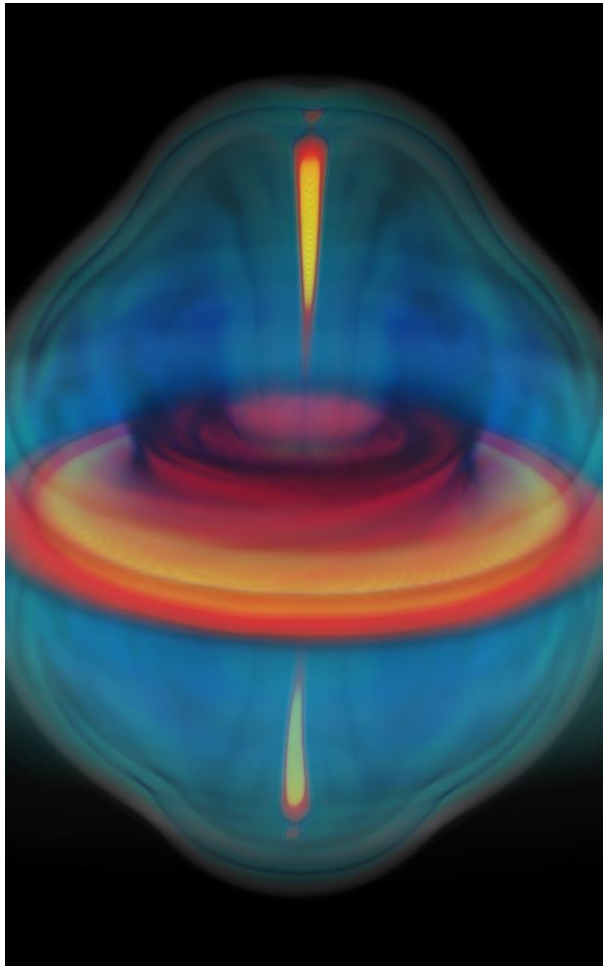
Science Case: Galaxy Mergers

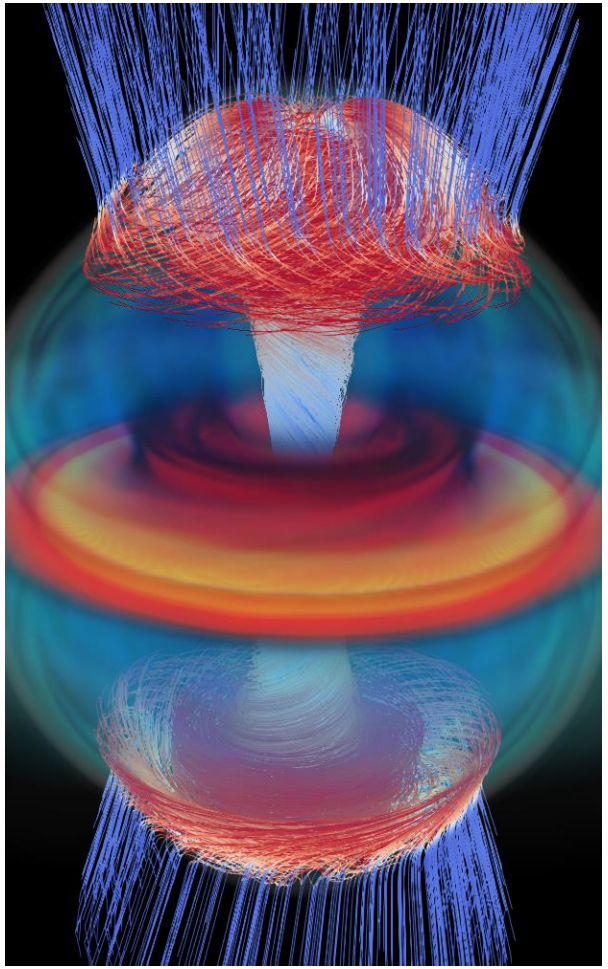
- Major merger between two galaxies, important for galaxy evolution and magnetic fields
- **Modules:** gravity, SPMHD, radiative cooling & star formation and feedback
- **Challenges:** Highly clustered data, large dynamical scales, complex and interconnected physics
- **Why exascale?** Capture the amplification of magnetic field from turbulent dynamos and galactic-scale dynamos



Science Case: Magnetized Cloud Collapse

- Early stage of the star formation process
- **Modules:** Gravity + SPMHD
- **Challenges:** Large-span of scales and tough test for MHD (divergence control and momentum conservation)
- **Why exascale:** to better simulating star formation, fluid instabilities in jets



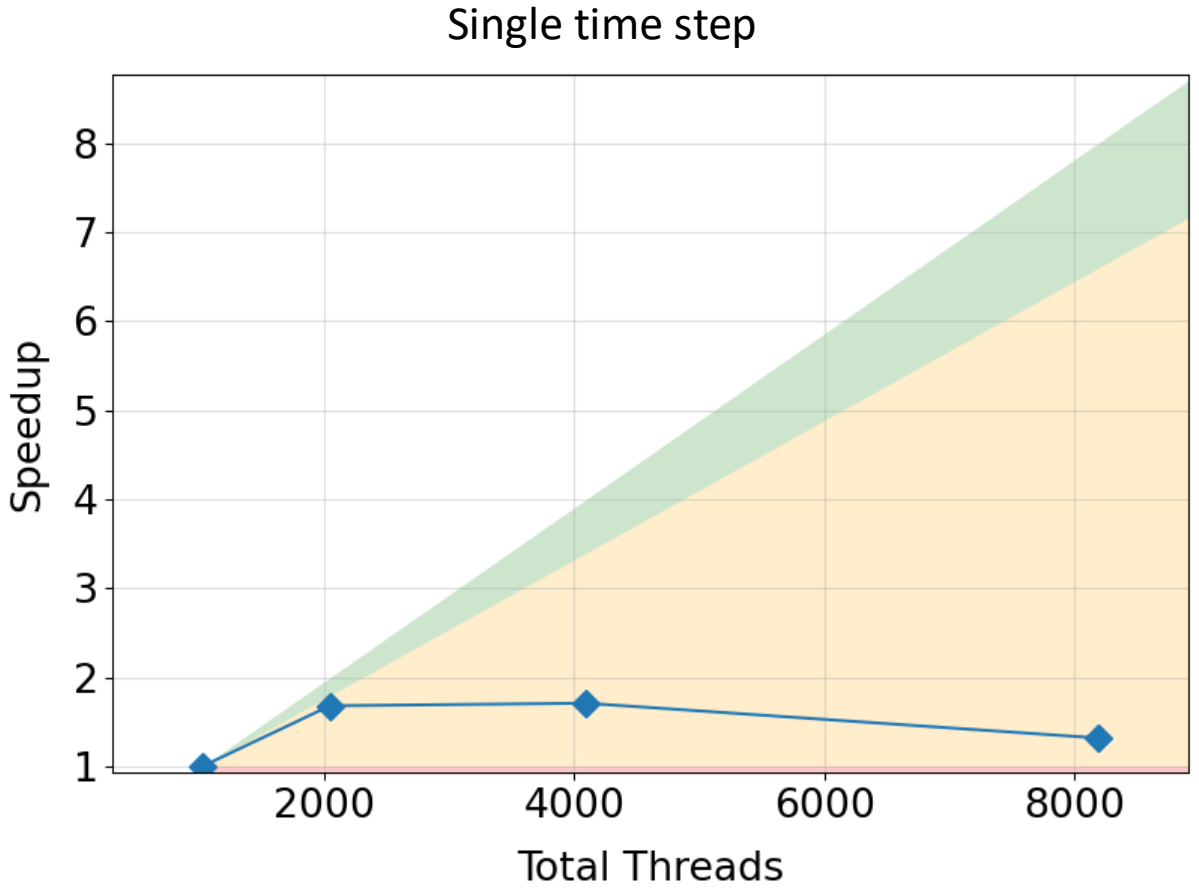


Objectives within SPACE for ChaNGa

1. Improve communication imbalance in full physics clustered data sets
2. Improve GPU version of code and start GPU offloading SPMHD and radiative cooling
3. Generate an extensive test suite and CI/CD for ChaNGa
4. Port and scale on all EuroHPC clusters
5. Explore improvements to vectorization

Communication Imbalance in ChaNGa

- The highly-clustered merger simulation leads to some tree pieces being requested much more than others, leading to significant delay in receiving messages (worsening with increasing threads)

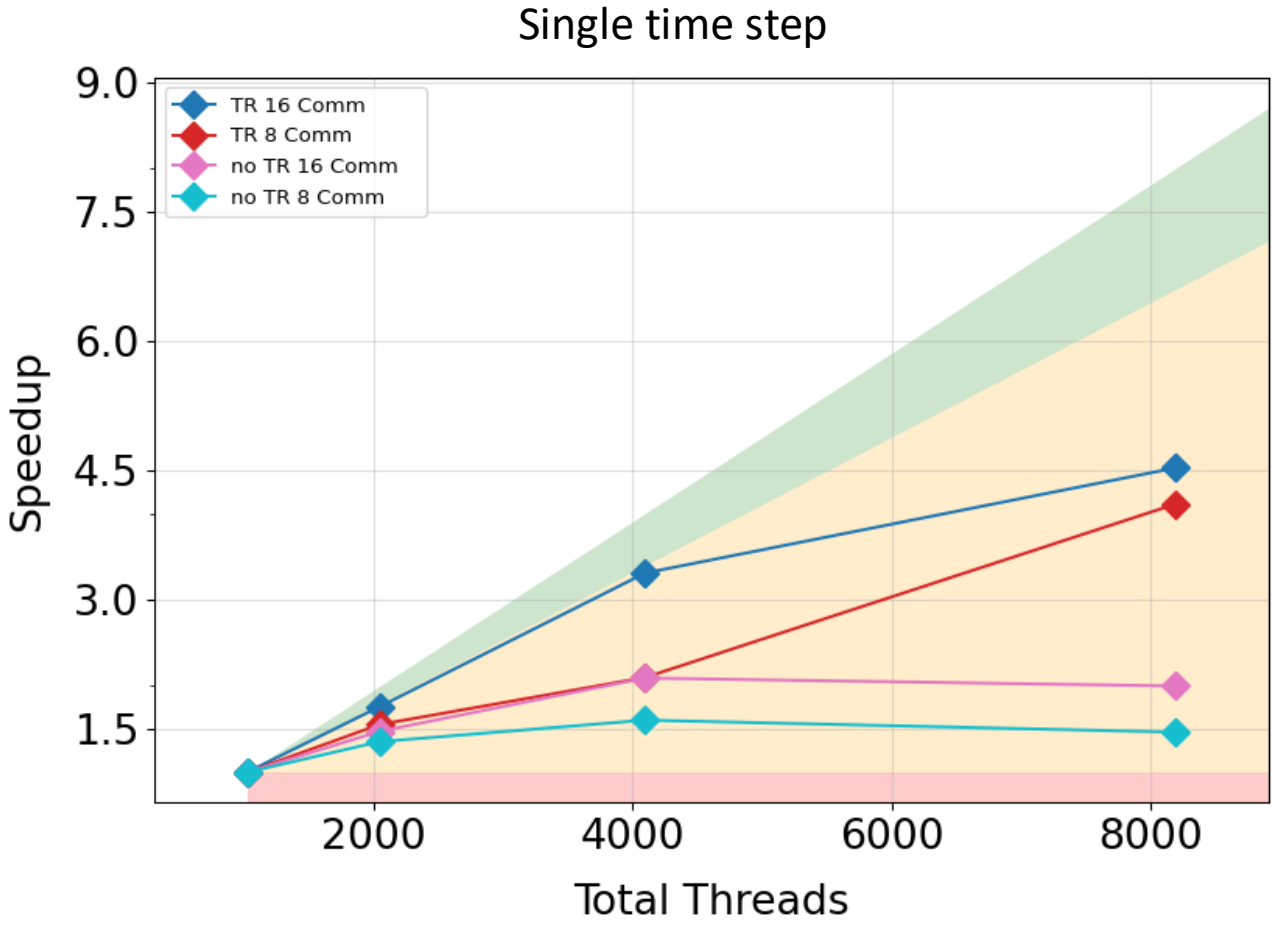


Optimization – Tree piece replication

- Tree piece replication

Replicate the tree node information on multiple processors ensuring that no single processor becomes overloaded

When a tree piece needs to request for a remote node, it chooses randomly one of the tree piece proxies to send the request to.

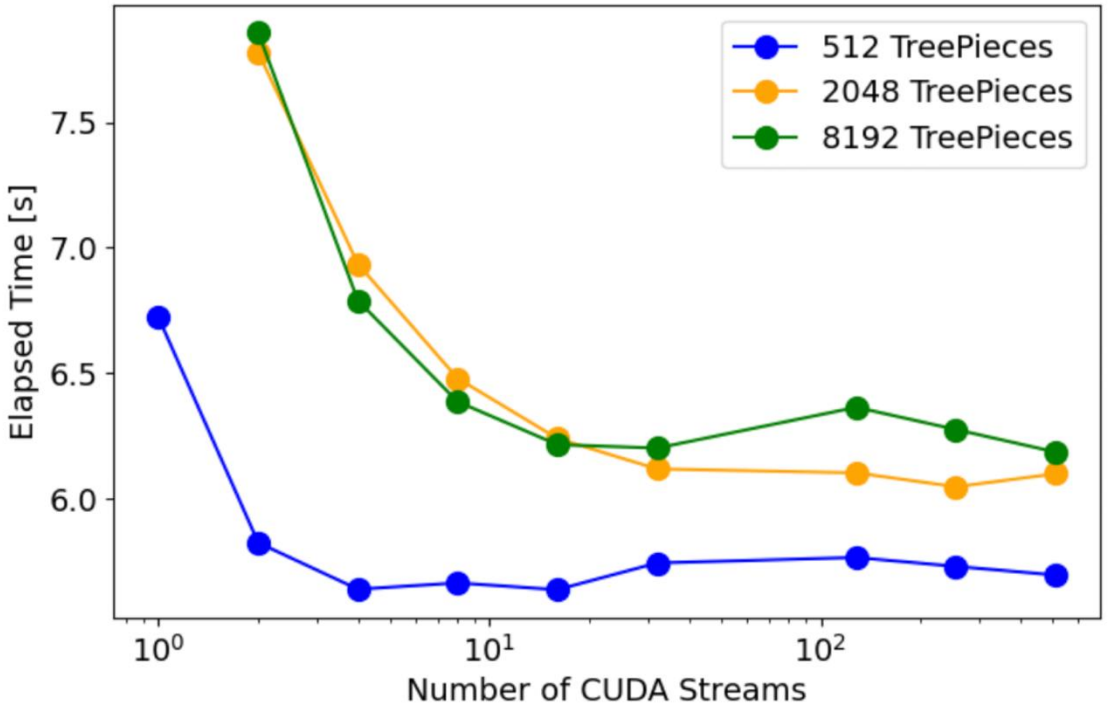


GPU version improvements

- Removal of deprecated workRequest API

  (limited to 1 CUDA stream/PE)
- Streams and GPU memory managed directly by the application instead

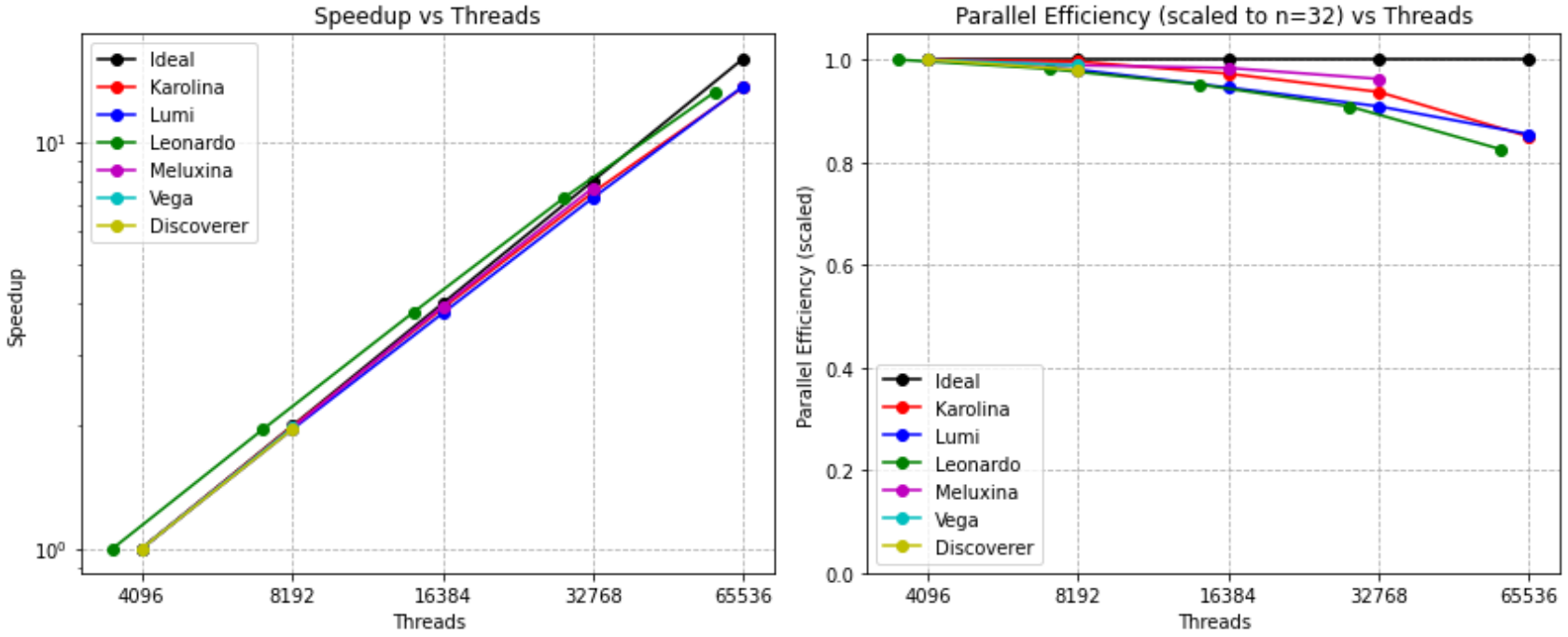
- Switch to do local tree walk on CPU when number of active particle falls below certain threshold
- GPU offload of radiative cooling
  (20x speedup, single node, single gpu)



Spencer Wallace

Porting and scaling on EuroHPC systems

ChaNGa has been ported to all EuroHPC systems except Deucalion (compiled and run on single node) and we have run scaling tests on most of these systems



ChaNGa CI/CD and testsuite

1. ChaNGa is publicly available and simple CI/CD exists (https://github.com/N-BodyShop/changa)

2. Extensive testsuite developed for ChaNGa (https://github.com/robertwissing/testsuite)

Summary

- Tree piece replication improves scaling in highly clustered data sets
- Improved GPU version
- Extensive test suite available at https://github.com/robertwissing/testsuite
- Code ported and scales for many EuroHPC clusters
- Public git at https://github.com/N-BodyShop/changa

Future plans

- Further testing of tree piece replication and exploring other improvements to communication
- GPU offloading the SPMHD
- Explore vectorization of SPMHD module

Acknowledgement & Disclaimer

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Belgium, Czech Republic, France, Germany, Greece, Italy, Norway, and Spain under grant agreement No 101093441.

Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European High Performance Computing Joint Undertaking (JU) and Belgium, Czech Republic, France, Germany, Greece, Italy, Norway, and Spain. Neither the European Union nor the granting authority can be held responsible for them